UNIVERSITY OF COPENHAGEN
FACULTY OF SCIENCE

Quantum Filtering Using Physics-Informed Neural Networks

By Mads Christian Olsen Barslev, Thorbjørn Ramløv & Lauge Lindegård Weber

Supervised by Professor Klaus Mølmer (klaus.molmer@nbi.ku.dk)
Cosupervised by Ph.D. fellow Luuk Jan-Heimen van den Berg (luuk.berg@nbi.ku.dk)

University of Copenhagen
Faculty of Science
Bachelor's Degree in Physics

Copenhagen N, 10 June 2026

Abstract

Using the Stochastic Master Equation (SME), we simulate weak measurements on a Rabi qubit system to obtain a measurement record $J(t)$. We build a neural network-based model that predicts the conditioned state evolution $\rho(t)$, given only the associated record $J(t)$ and the result of a final projective measurement. The latter quantity serves as the target that the model's prediction is essentially designed to match. The model fundamentally consists of two parts: an LSTM, which implements memory effects in the evolution, and a Kraus formalism, which ensures physical validity of the reconstructed state dynamics. With reference to the density matrix, the network performs the best on the very element that gives rise the measurement record. Although not directly optimized against the other dimensions, the model still makes partially valid predictions, which are generally biased towards unitary evolution. The model cannot capture variation in Rabi frequencies between trajectories, which presumably can be explained by a low signal-to-noise ratio in $J(t)$. Estimation of the physical parameters — by numerically fitting an SME to the predicted state trajectory — is of poor quality, indicating possible incompatibility between the model and the SME. We suggest revising the role of Kraus operators in the model, but the predictive power of the model is still pronounced and possibly applicable to real experimental situations.

Acknowledgements

We are thankful to Prof. Klaus Mølmer (klaus.molmer@nbi.ku.dk) for supervising this bachelor project; for taking his time to share his passion and extensive knowledge of quantum mechanics with us, and for listening to our ideas and letting us set the direction of the project.

Many thanks also to Luuk Jan-Heimen van den Berg (luuk.berg@nbi.ku.dk) for co-supervising the project; for his inputs on technical matters, and for his kind willingness to help with theory and other questions.

Contents

1 Introduction

Quantum technologies are promising across different domains, such as quantum sensing and quantum computing. The latter has big potential in tackling problems where the necessary amount of processing is unattainable even for the best classical supercomputers. One example of such a problem would be the simulation of larger quantum systems, which, if solved, will scale innovation across material science, chemistry, and pharmacy [10].

Such technological implementation requires an extensive understanding of the behaviour of quantum systems. In some scenarios, it is useful to act on a quantum system in real time to influence it in a desired way — so called quantum feedback control. An example is continuous quantum error correction for quantum computing [22]. Quantum feedback control is done by weakly and continuously measuring a system, and then using this measurement record to best infer the current state of the system. The best estimate of the state is then used to inform the feedback on the system.

A natural way to do this state estimation is to assume the dynamics of the system, and then use a mathematical map to infer the state from the measurement record, such as in [34]. Meanwhile, the increase in computing capacity of classical computers have made machine learning very successful across various domains. One big advantage using this approach is that it requires minimal assumptions. Given sufficient structure, networks are able to learn statistical dependencies in data without complete knowledge of the underlying system dynamics.

In this Bachelor Project, we use a machine learning approach in estimating the state of a quantum system from a measurement record with inherent quantum noise. This has been done in various ways before, notably in [7] by Flurin et al., who applied a recurrent neural network on a qubit in an experimental setting. In absence of experimental measurement records, the data used in this project is simulated, which in return enables a more thorough comparison between the reconstructed and the true state dynamics, because the latter is only realisable in a simulated environment.

In 'Theoretical Framework', we lay out basic theory regarding continuous measurements as well as a short introduction to neural networks. In 'Model Overview', we describe the (simulated) physical system, and how the model is structured and trained. In 'Results', we go through the results obtained by applying the trained model on simulated data. Our findings show that whilst the model often-times accurately reconstructs the state, it also fails to infer important dynamics (and parameters), that are required to construct an accurate state. These results are discussed, and we provide some probable explanations for the model's behaviour. In the 'Discussion', we motivate our chosen model structure through the literature and suggest improvements to it. In the 'Conclusion', we summarise our main findings: how well the model reconstructs the conditional state, and what the parameters inferred from its predictions reveal about extracting per-trajectory information from the measurement record. In 'Perspectives', we summarise where we stand in relation to the literature and where our model might contribute.

All Python code used to generate the results in this project is available at `https://github.com/madschr8/quantum_filtering.git`

Theoretical Framework

2

2.1 From closed to open quantum systems

In closed system quantum mechanics, a quantum state is described by a vector $|\psi\rangle$ in Hilbert space. All systems interact with their environment to some degree, and to describe this openness of a system, one must introduce the density matrix ρ as a description of the state ([24] sec. 2.4.2). This description allows for a distinction between classical probabilities and the concept of a quantum superposition, which is necessary to capture the phenomenon of *decoherence* — the loss of quantum properties due to interaction with an environment ([33] p. 97). One can imagine a scenario where the uncertainty of a state is conditioned on a probability in a classical sense, say, because the preparation in either spin $|\uparrow\rangle$ or $|\downarrow\rangle$ is determined by a coin flip (or some unknown interaction with an environment). The density matrix would then be constructed as

$$\rho = \frac{1}{2}|\uparrow\rangle\langle\uparrow| + \frac{1}{2}|\downarrow\rangle\langle\downarrow| = \frac{1}{2}\begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix} + \frac{1}{2}\begin{bmatrix}0 & 0\\ 0 & 1\end{bmatrix} = \begin{bmatrix}\frac{1}{2} & 0\\ 0 & \frac{1}{2}\end{bmatrix} \tag{2.1}$$

Such a state is referred to as a *mixed state* [23]. The density matrix diagonal elements ρ_{00} and ρ_{11} are the populations — the probabilities of finding the qubit in the ground or excited state, $|g\rangle$ and $|e\rangle$ ($|0\rangle$ and $|1\rangle$, equivalently). The statistical interpretation enforces two constraints on ρ [20]: First, the probabilities must sum to unity, corresponding to $\mathrm{Tr}[\rho] = \rho_{00} + \rho_{11} = 1$; second, ρ must be *positive semi-definite*. The second condition is met if and only if ρ is Hermitian ($\rho = \rho^\dagger$) and has non-negative, real eigenvalues. Hermiticity leads to $\rho_{01}^* = \rho_{10}$ for the off-diagonals. These represent the amount of coherence in the state. In the above example, there is zero coherence, and the measurement outcome is thus a result of classical probabilities only (and not the result of a quantum superposition). There are effectively three independent variables in ρ, and therefore three sets of projective measurements are needed to fully reconstruct a density matrix from experiment ([24] p. 71).

2.1.1 Dephasing and energy relaxation

There are two key phenomena that distinguish closed quantum systems from open ones: *energy relaxation* and *dephasing* ([24] p. 34). They are both subcategories of decoherence that in this aspect covers the mixing of a state due to the reduction of its off-diagonal (coherence) elements ([14] s. 9). Dephasing is the result of fluctuations in the phase between the two qubit populations due to environmental noise. Averaging over the uncertain phase decays the off-diagonal elements, while leaving the diagonal ones unchanged. Relaxation is the transition from the excited to the ground state, $|e\rangle \to |g\rangle$, driven by an unmonitored energy interaction with the environment: Whether a relaxation event has occurred, reduces our certainty about the populations. This loss of information decoheres the state, reducing the off-diagonal elements while also redistributing the diagonal entries ([24] sec. 2.4.2).

2.1.2 The Bloch sphere visualisation

A useful representation of a qubit state is the *Bloch vector*: By expressing the 2-level quantum state in terms of the Pauli operators ($\hat{\sigma}_x$, $\hat{\sigma}_y$, $\hat{\sigma}_z$) and the identity matrix, $\hat{I}$, we obtain a real 3-dimensional vector $\vec{r}$ constrained to (*pure state*) or within (*mixed state*) a unit sphere called the *Bloch sphere* ([33] sec. 3.3):

$$\begin{aligned}\rho &= \tfrac{1}{2}\left(\hat{I} + \vec{r}\cdot\vec{\sigma}\right), \qquad |\vec{r}| \leq 1,\\ \vec{r} &= \left(\langle\hat{\sigma}_x\rangle, \langle\hat{\sigma}_y\rangle, \langle\hat{\sigma}_z\rangle\right),\\ \vec{\sigma} &= \left(\hat{\sigma}_x, \hat{\sigma}_y, \hat{\sigma}_z\right).\end{aligned}$$

This representation is useful since it allows for a geometric visualisation of the effects of operators on ρ for a two-level system. It also allows for immediate reading of $\langle\hat{\sigma}_{b_i}\rangle$ since the four matrices are mutually orthogonal. Equivalently ([25] sec. 8.3.2), the parametrisation can be expressed by the following:

$$\rho_{00} = \tfrac{1}{2}\left(1 + \langle\hat{\sigma}_z\rangle\right), \quad \mathrm{Re}(\rho_{01}) = \tfrac{1}{2}\langle\hat{\sigma}_x\rangle, \quad \mathrm{Im}(\rho_{01}) = -\tfrac{1}{2}\langle\hat{\sigma}_y\rangle, \tag{2.2}$$

giving the components of the Bloch vector along the z, x and y axes, respectively. ρ_{00} runs from 0 to 1, while $\mathrm{Re}(\rho_{01})$ and $\mathrm{Im}(\rho_{01})$ span from $-1/2$ to $+1/2$ — however not independently due to the vector norm boundary of 1.

The later choice of parametrisation will be the primary visual framework of the project, and — to ensure clarity — it should be explicitly said that the north pole of the Bloch sphere represents the ground state, $|g\rangle$.

2.2 Continuous Measurement

The traditional notion of a measurement of a quantum system is a projective measurement: A specific physical quantity has an associated operator, and when that operator is applied to an e.g. superposition state, the state collapses into one of the eigenvalues of that operator and destroys the superposition. In the measurement process, one gains information about the state. However, not all ways of measuring result in a virtually instantaneous total collapse of the state. Instead, the state partially collapses over time as determined by the *measurement strength* (loosely, how fast the state collapses). If this measurement strength is low, the state collapses only slowly and partially (since information gain per measurement instant is low). This gradual measurement is what we call a *weak measurement* ([14] p. 71). This is useful in cases where the properties of the superposition are essential — as in quantum computing — but one still wants to extract information about the system. Analogous to the wave-particle duality of light, "the behaviour of a quantum emitter depends on how we detect its emission" ([24] p. 4). If we detect the wave nature of light, as opposed to the particle nature, the quantum emitter (e.g. qubit) no longer makes a sudden jump. Rather, it takes on diffusive dynamics and follows a continuous path, known as a *quantum trajectory*. We will focus on the kind of weak measurement known as *homodyne* measurement:

We consider the qubit to be coupled to a cavity in a way such that the cavity's resonance frequency is dependent on the qubit population. The cavity is populated for a time k^{-1} ([24] sec. 3.1) by a small amount of photons from a microwave (MW) probe created by a Local Oscillator (LO), and the phases of these photons are shifted due to the coupling to the qubit population, so when the photons leave the cavity, they essentially carry information about the populations ($\langle\hat{\sigma}_z\rangle$) (a signal) ([24] sec. 3.4.3). The LO output is split: it does not send all of its photons as a probe to the cavity, but also leaves some as a fixed phase (a copy) of the original LO probe. The photons from the cavity are then allowed to interfere with the LO copy, and since the copy-photons are at the same (*homo*) frequency as the signal photons (note, only the phase changes), the output signal is a slowly-varying signal, whose amplitude is determined by the phase difference between the signal and copy photons with additional white noise by vacuum fluctuations ([24] sec. 2.1.1) and amplifier noise ([24] sec. 3.4). Since the LO copy has a fixed phase, the output amplitude

depends on the signal phase, which is dependant on the qubit state ([24] sec. 3.3.3). The resulting signal (current) is a white-noise signal that tracks $\langle\hat{\sigma}_z\rangle$ and thus the diagonal elements (*populations)* of the density matrix. Inferring the state evolution from a measurement current is called *Quantum Filtering* ([33] p. 309). This measurement scheme can be expanded to include measurements of $\langle\hat{\sigma}_x\rangle$ and $\langle\hat{\sigma}_y\rangle$.

In all quantum measurement processes, there is an inescapable quantum noise. According to [24] p. 15, this quantum noise can be interpreted as originating from the vacuum fluctuations in the MW field: "[...] even an empty cavity has some amount of energy and [the] electric and magnetic field fluctuate around zero" ([24] p. 11). Thus, there is an inherent uncertainty in how the MW field interacts with the qubit, and this manifests as the white noise in the measurement signal. For true experimental setups, there will naturally be additional sources of noise and decoherence but one cannot beat this fundamental quantum noise.

2.3 Stochastic Master Equation (SME)

The SME describes the temporal evolution of an open quantum system conditioned on a noisy measurement record [18].

$$d\rho(t) = \underbrace{-i[H,\rho(t)]dt}_{\text{von Neumann}} + \underbrace{\Sigma_c\mathcal{L}[c]\rho(t)dt}_{\text{Lindblad dissipator}} + \underbrace{\mathcal{H}[c]\rho(t)dW(t)}_{\text{measurement update}} \tag{2.3}$$

The first term is the *von Neumann equation*. It describes the unitary time evolution of ρ as a closed system governed only by $\hat{H}$ (and is thus the density matrix equivalent of the Schrödinger Equation). The second term is the Lindblad dissipation term that captures the unconditional dynamics of ρ. The third term is the conditional update of ρ based on the measurement current. $\mathcal{L}$ and $\mathcal{H}$ are so-called *super-operators* (conditioned on the measurement operator $\hat{c}$) acting on ρ ([33] p. 21). In this context, since we do not go deep into the maths, it is enough to think of them as a notational convenience and know that the terms depend in some way on the operators.

When homodyne measurement is conducted on a system, it results in a noisy current $J(t)$ ([24] eq. 4.3.3), which can be used to update the state:

$$J(t) = 2\sqrt{\kappa\eta}\langle\hat{\sigma}_{b_i}\rangle + \frac{dW}{dt} \quad , \quad b_i \in \{x,y,z\} \tag{2.4}$$

The noise is modelled mathematically by the *Wiener-increment*, dW. It is a random Gaussian variable with $\langle dW\rangle = 0$ and $\langle dW^2\rangle = dt$. This means that the Wiener-increment scales as $dW \sim \sqrt{dt}$. In this lies a problem: the higher time scale resolution with which one wants to know $\langle\sigma_{b_i}\rangle$, the more present the noise becomes in the signal.

The measurement *back-action* itself becomes important for the *conditional* evolution of the state: it is the price one must pay when extracting information from the system, nudging the state toward an eigenstate of the measurement operator. This is described by the measurement update-term in the SME. By expressing dW in terms of $J(t)$, one can reconstruct the state evolution from the measurement record.

Depending on the chosen operators, the measurement operator contracts the components of $\vec{r}$ that are transverse to the measurement axis — Lindblad type dynamics — and stochastically pulls the longitudinal component toward one of its eigenstates ([14] sec. 5.4 p.117). This type of measurement has an *efficiency*, η, which tells us how much of the possible information during a measurement actually gets recorded. An $\eta < 1$ efficiency can be implemented by independently measuring concurrently with a second measurement operator in the same direction of the same strength as the first, but not record the measurement ([24] sec. 4.3.3). When we assume $\eta = 1$, we have that the ratio between the measurement rate and the dephasing rate is unity, thus we measure as fast as the state dephases ([14] sec. 7.2). Building on the behaviour of the ρ elements discussed in chap. 2.1.2, a measurement changes the populations, but most practical schemes

reduce the off-diagonal elements as well. Since measurement efficiency is rarely 100%, mixing appears (information is lost), resulting in what is called *measurement-induced dephasing* ([24] sec. 4.3.3).

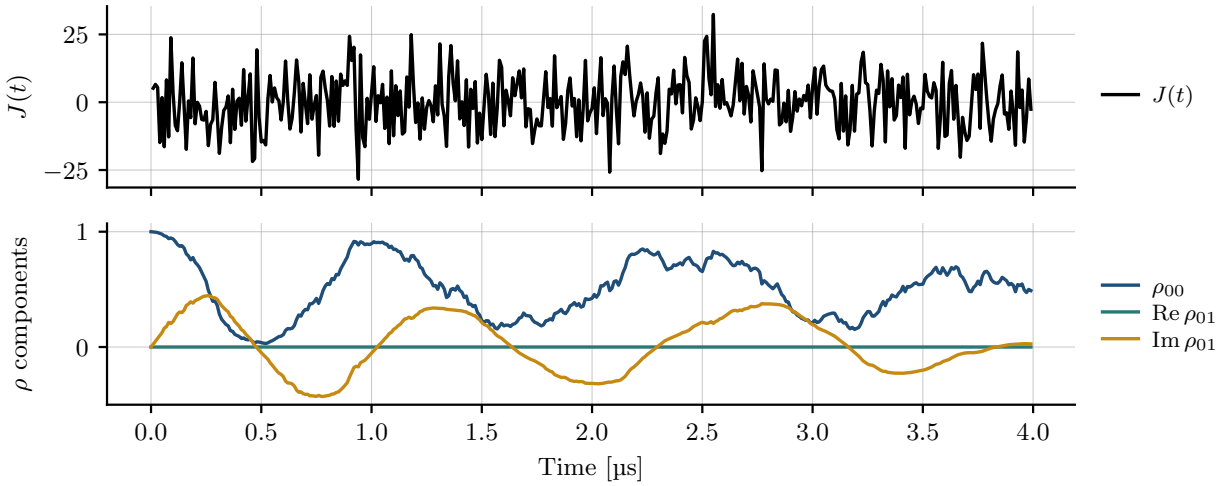

Figure 2.1: Example of a realisation of a quantum trajectory. The stochastic noise in $J(t)$ (partly) determines the state evolution (along with the Hamiltonian and other Lindblad operators. See Section 3 for a total breakdown of the system). The trajectory is generated using the Python library QuTiP [18] (see Sec. 3).

2.4 Kraus Operators

A very general way to describe operations on a quantum state is through the framework of *quantum channels*. An operation on a quantum state can be anything that physically interacts with and changes a state — for instance measurements. We restrict ourselves to *Markovian* dynamics, which means that the state evolution is dependent only on the current state (and not previous states).

A more exact definition of a quantum channel is that it is a linear map Φ that maps density matrices to density matrices, $\rho \to \Phi(\rho)$ [20]. Since the quantum channel maps from one valid ρ to another valid ρ, it must preserve the statistical interpretation: that is $\mathrm{Tr}(\rho) = 1$, and ρ being positive semi-definite. In other words, Φ must be *Completely Positive Trace Preserving* (CPTP) to be a valid map describing a quantum channel.

Kraus operators are one way of representing quantum channels using matrices and the one that we will use. Any quantum channel can be expressed with Kraus operators in this way [19]:

$$\Phi(\rho) = \sum_{i=0}^{N-1} K_i \rho K_i^\dagger. \tag{2.5}$$

In order for Φ to be trace preserving (TP), we demand that

$$\mathrm{Tr}(\Phi(\rho)) = \mathrm{Tr}\left(\sum_i K_i \rho K_i^\dagger\right) = \sum_i \mathrm{Tr}(K_i \rho K_i^\dagger) = \sum_i \mathrm{Tr}(K_i^\dagger K_i \rho) = \mathrm{Tr}\left(\underbrace{\sum_i K_i^\dagger K_i}_{\hat{I}}\, \rho\right) = \mathrm{Tr}(\rho). \tag{2.6}$$

The above equation holds only if the following criteria is met

$$\sum_{i=0}^{N-1} K_i^\dagger K_i = I. \tag{2.7}$$

To be completely positive (CP), $\Phi(\rho)$ must be positive semidefinite given that ρ was positive semidefinite ([19], page "Quantum Channel Basics"). [1] This property is automatically ensured from Eq. 2.5 ([19]).

To represent a measurement using Kraus operators, one constructs a set $\{\hat{E}_0, ...\hat{E}_J\}$ of measurement operators, one for each possible outcome j of the measurement

$$\hat{E}_j = \hat{K}_j^\dagger \hat{K}_j. \tag{2.8}$$

This construction ensures that $\hat{E}_j$ is positive semidefinite. The probability of the measurement outcome j is given by

$$P_j = \text{Tr}[\hat{E}_j \rho] = \text{Tr}[\hat{K}_j \rho \hat{K}_j^\dagger]. \tag{2.9}$$

Since both $\hat{E}_j$ and ρ are positive semidefinite, P_j is ensured to be a positive value: "[...] the trace of the product of any two positive semidefinite matrices is always nonnegative" ([20] sec. "Mathematical formulations of measurement"). Using Kraus operators, we can construct the post-measurement state of $\rho^{(j)}$ by conditioning the initial state ρ_0 on the measurement outcome represented by j ([14] sec. 3.6). Generally, this means that every outcome of measurement j has a distinct quantum state to which the current state is mapped ([14] sec. 3.4). Thus, the normalized conditional state given a measurement outcome j is constructed as

$$\rho(t_k) \longrightarrow \rho^{(j)}(t_{k+1}) = \frac{\hat{K}_j \rho(t_k) \hat{K}_j^\dagger}{\text{Tr}[\hat{K}_j \rho(t_k) \hat{K}_j^\dagger]}. \tag{2.10}$$

Another unavoidable part of realistic measurement is that where the meter state cannot be recorded or is averaged over. This can be investigated by an *ensemble average* of the conditional $\rho^{(j)}$, or over Kraus operators acting on ρ, representing the specific type of unrecorded measurements (as described by the Lindblad operators in section 2.2) ([14] sec. 3.7)

$$\rho(t_k) \longrightarrow \rho(t_{k+1}) = \sum_j P_j \rho^{(j)} = \sum_j \hat{K}_j \rho \hat{K}_j^\dagger. \tag{2.11}$$

Kraus operators thus provide a general way to evolve a state after a measurement, both in case of knowing the measurement result (a *monitored channel*) and not knowing it (an *unmonitored channel*). Hence, they are used to structure our neural network. Since we are doing homodyne measurements, we would at first glance need infinitely many $\hat{E}$-operators, since the measurement outcome spans a continuum of possibilities [33] eq. (1.88). Luckily, in the context of the Kraus formalism on a two-level system, every imaginable Markovian evolution of ρ can effectively be captured using four Kraus operators (Theorem 8.3 in [25]).

2.5 Neural Networks and LSTM

A neural network consists of interconnected neurons, each characterised by an activation value y and its connections to other neurons. A typical network structure consists of different layers ℓ. Each layer consists of a number of neurons with activation values in vector form $\vec{y}^\ell$ (superscript is index). Each neuron is connected to all other neurons in the previous and next layer. The layers are updated in sequence, starting with some input vector $\vec{y}^0$ according to an update rule ([17] chap. 2)

$$\vec{y}^{\ell+1} = f(\mathbf{W}^\ell \vec{y}^\ell + \vec{b}^\ell). \tag{2.12}$$

The hope is that with the appropriate weight matrix $\mathbf{W}^\ell$ and bias vector $\vec{b}^\ell$ for each layer, one can map the input vector to an output vector in a desirable way. Usually, the activation function f is a non-linear function that restricts $\vec{y}$ to values in the range $[-1, 1]$.

Recurrent Neural Networks (RRNs) expand the above idea to suit temporal datasets ([1] sec. II.B). The general idea is to have an identical layered neural network for each time step t in the dataset. The network

[1] The *complete* part of CP actually makes it a stronger requirement than what is presented here. See [20]

then processes one time step at a time, goes through all the layers for that time step, before passing the result to the next time step. The update then depends not only on $\vec{y}_t^{\,0}$ (the input vector at t), but also on all previous time steps. That means the network effectively has memory throughout the data sequence.

The *Long Short Term Memory-network* (LSTM) is one effective way to implement such a RNN. The cell state C_t and hidden state h_t are the part of the LSTM that makes it suitable for learning time dependencies in the $J(t)$ signal. For a breakdown of the LSTM structure, see Appendix A.

When the network structure is defined, the problem consists in optimizing all the $\mathbf{W}^\ell$ and $\vec{b}^\ell$ against some defined loss function. This is referred to as *back-propagation* ([17] chap. 4) which is how the model is trained. The state of each output neuron is compared to the loss function to determine the error (for a loss of 0, the network predicted the desired output perfectly). If the output state was, say, too high, then the weights connecting to that neuron are adjusted in the direction that lowers its output. This is achieved by updating each weight according to the existing connection and the error (using some non-linear function). The constant of proportionality determining the size of the weight update is called *the learning rate*.

Model Overview

3

The central task of this thesis is to reconstruct the conditional state evolution $\rho(t)$ of a continuously monitored qubit from its homodyne measurement record $J(t)$. A usual approach would be to assume the physical model (i.e., the SME) and measure all relevant physical parameters, and then use an analytical map (also called a quantum filter in this context) to find the conditional state evolution (see for example [34]). These maps require *a complete specification of the underlying model* [34]: every operator acting on the system together with the exact physical parameters must be specified in order to make use of a dynamical map. In practice, there is often a discrepancy between the actual dynamics of the system and the best available model of those dynamics. Another way to put this is that another underlying model — or different values of the physical parameters — could potentially better capture the actual dynamics of the system and result in a better conditional evolution. In the end, this would lead to better predictions; cf., the possible use cases of quantum feedback control mentioned in Introduction.

In this context, neural networks are powerful tools. Even if a network does not provide understanding of a physical system, it may still be extremely useful in cases of quantum feedback control, where a good prediction is of highest interest. Since NNs can capture statistical dependencies in data of all sorts, they have the potential to predict the conditional dynamics more efficiently than the analytical maps, in cases where the knowledge of the system dynamics is incomplete.

Drawing on the concepts laid out in Theoretical Framework, we now explain the way our model is structured and the ideas behind the design choices.

3.1 Simulating measurement data

We use the QuTip [18] package in `Python` to simulate measurement data, allowing us to explore a wide range of system specifications such as parameter values, monitored and unmonitored operators. Simulating the data in this manner also has the advantage of providing a ρ_{true} that can eventually be used to evaluate the performance of the model.

3.1.1 System operators and subsequent dynamics

After some deliberation, we arrived at a suitably challenging system from which to generate training data. The system is a qubit subject to a *Rabi Hamiltonian*

$$\hat{H}_{R,i} = \frac{\Omega_{R,i}}{2}\hat{\sigma}_x, \tag{3.1}$$

which has the effect of rotating the Bloch-vector in the yz-plane, so that the population varies sinusoidally as can be seen on the trajectory example in Figure 2.1.

We generate data in two regimes. In the *fixed* regime, the Rabi frequency is identical across all trajectories, $\Omega_{R,i} = \Omega_R$ for all i. In the *varying* regime, the frequency $\Omega_{R,i}$ is drawn independently and uniformly for each trajectory from a distribution centered on the nominal value Ω_R:

$$\Omega_{R,i} \sim \mathcal{U}(0.8\,\Omega_R,\ 1.2\,\Omega_R). \tag{3.2}$$

Here, $\{\Omega_{R,i}\}_{i=1}^{N}$ are the independent draws for each of the N trajectories, and $\mathcal{U}$ is a uniform distribution.

The qubit is initialized in the ground state — the lowest-energy state in the absence of the drive — corresponding to preparing the system in a known state before measurement begins:

$$|\psi_0\rangle = |g\rangle \quad \Rightarrow \quad \rho_0 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

The qubit is continuously monitored by a *measurement operator* along one of the three Pauli axes, with the axis b_i chosen independently for each trajectory

$$\hat{C}_{\text{mon},i} = \sqrt{\eta\,\kappa}\,\hat{\sigma}_{b_i}, \qquad b_i \in \{x, y, z\},$$

where κ ($[\kappa] = \text{s}^{-1}$) is the *measurement strength* and η the *measurement efficiency*. Each trajectory i is thus characterized by the pair $(\Omega_{R,i},\, b_i)$. We imagine the realistic case $\eta < 1$ in which only a fraction η of the information about the observable reaches the detector ([14] sec. 4.2). To physically motivate the missing fraction, we introduce a second, unmonitored operator along the same axis b_i, representing the portion of information that escapes through an inaccessible channel ([33] sec. 4.9.3)

$$\hat{C}_{\text{loss},i} = \sqrt{(1-\eta)\,\kappa}\,\hat{\sigma}_{b_i}.$$

Together, $\hat{C}_{\text{mon},i}$ and $\hat{C}_{\text{loss},i}$ split the same measurement strength κ into a monitored fraction η — with *measurement rate* $\eta\kappa$ — and an unmonitored fraction $1-\eta$ — with rate $(1-\eta)\kappa$ — and thus reproduce the unconditional Lindblad dynamics that would arise from a single operator $\sqrt{\kappa}\,\hat{\sigma}_{b_i}$ on this axis. Only $\hat{C}_{\text{mon},i}$ contributes to the recorded measurement current $J(t)$ ([33] sec. 6.4).

To illustrate that the qubit also relaxes toward the ground state through coupling to a reservoir, we include an unmonitored decay operator written in terms of the lowering operator $\hat{\sigma}_-$ as inspired by Gambetta et al. [8]

$$\hat{C}_{\text{decay}} = \sqrt{\gamma_d}\,\hat{\sigma}_-,$$

where γ_d ($[\gamma_d] = s^{-1}$) is the spontaneous emission rate. Whenever the Rabi drive populates the excited state $|e\rangle$, this operator relaxes the qubit back toward $|g\rangle$, capturing the energy relaxation that would arise from coupling to a cavity or other reservoir in a realistic device ([33] sec. 5.3.1).

Although both $\hat{C}_{\text{loss}}$ and $\hat{C}_{\text{decay}}$ are unmonitored and both contribute to information loss, they do so through physically distinct channels. In the limit $\eta \to 1$, $\hat{C}_{\text{loss}}$ vanishes and all of the information from this channel ends up in $J(t)$ ([14] sec. 5.4). $\hat{C}_{\text{decay}}$, by contrast, represents spontaneous emission of energy to a reservoir and would not contribute to $J(t)$ even at perfect detection efficiency.

Several effects therefore compete to set the conditional dynamics. The Hamiltonian $\hat{H}_R$ rotates the Bloch vector in the yz-plane, while the measurement operator $\hat{C}_{\text{mon}}$ feeds information about the measured observable into $J(t)$ and pulls the conditional state toward one of its own eigenstates. When the Hamiltonian and the measurement do not commute — which is the case for two of the three Pauli choices ($\hat{\sigma}_y$ and $\hat{\sigma}_z$) — these two effects act in different directions, and the resulting trajectory reflects a balance between them (see the results of Flurin et al. [7]). In the case of the $\hat{\sigma}_x$ measurement, the measurement operator commutes with the Hamiltonian ($[\hat{H}, \hat{\sigma}_x] = 0$), thus there is no competition between the measurement back-action and the Hamiltonian.

3.1.2 Data Generation using QuTip

We employ QuTip's `smesolve` that — given an initial state, operators, and parameters (Table 3.1) — integrates the SME at each time step dt. For each timestep, a Wiener increment, dW, is drawn to determine $J(t)$ and further evovle $\rho(t)$ according to eqs. (2.3) & (2.4).

We generate N trajectories per measurement axis. For each trajectory, a new $\Omega_{R,i}$ is drawn from the uniform distribution and a single `smesolve` call produces $J(t)$ alongside the conditional $\rho(t)$. At the end of each trajectory (inspired by [7]), we perform a single projective measurement in the chosen Pauli basis. The binary outcome $y \in \{0, 1\}$ is drawn from a Bernoulli distribution with success probability $P_{\text{end}} = \text{Tr}[\hat{P}\,\rho(T)]$, where $\hat{P}$ is the projector onto the $+1$ eigenstate of the measurement axis.

For each trajectory i, we record the currents $J_i(t)$ and the final projective spin outcome $y_i \in \{0, 1\}$. These are measurements that would be obtainable in the lab, which is why we restrict the model to train and predict only on these. Since we simulate our measurements, we also have access to the true state evolution $\rho_i(t)$, the true projective probability $P_{\text{end,i}}$, and the true Rabi frequency[1]. Apart from a calibration of the Rabi frequency, these measures would not be obtainable in a real lab-situation, which is why we only use them to evaluate the performance of the model, and not for its training or prediction. Additionally, both sets have knowledge about the the basis $\hat{\sigma}_b$ in which the measurement was done. So the test and training sets are:

$$\mathcal{D}_{\text{train}} = \{(J_i(t),\, y_i,\, \hat{\sigma}_{b,i})\}_{i=1}^{N_{\text{train}}},$$
$$\mathcal{D}_{\text{test}} = \{(J_i(t),\, y_i,\, \Omega_{R,i},\, P_{\text{end},i},\, \rho_i(t)\,, \hat{\sigma}_{b,i})\}_{i=1}^{N_{\text{test}}},$$

`Parameter name` (& symbol)	Value	Description
`dt`	$0.01\mu s$	Num. integration resolution
`t_total` (T)	$4.0\mu s$	Total measurement time
`eta` (η)	0.30	Measurement efficiency
`GAMMA_DECAY` (γ_d)	0.1 MHz	Unmonitered relaxation rate
`KAPPA` (κ)	0.4 MHz	Measurement strength ([24] sec. 4.2.3 p. 78)
`OMEGA` (Ω_R)	$\mathcal{U}[0.8\,(2\pi \cdot 0.8)\,,\ 1.2(2\pi \cdot 0.8)]$ MHz	Rabi oscillation frequency drawn from uniform distribution $\mathcal{U}$

Table 3.1: Parameters used in simulation of measurement data. "Num." stands for Numerical.

The chosen numerical values of the parameters are important for the dynamics of the qubit — as apparent from e.g. Koolstra et al. [15]. We consider a system specified by the values in Table 3.1, with values mainly inspired by Flurin et al. [7]. The measurement strength, κ, is chosen low enough to avoid suppressing the Rabi oscillations and collapsing the state due to measurement (as described by [8] sec. VII).

While Flurin et al. employs a qubit relaxation time of $T_1 = 60\ \mu s$ equivalent to $\gamma_d = 0.0167$ MHz, we employ $T_1 = 10\ \mu$s ($\gamma_d = 0.1$ MHz), meaning the qubit relaxes ~ 6 times faster, which acts a challenge to the neural network due to increased decoherence. According to Naghiloo ([24] p. 84), the contribution from the energy relaxation time to the dephasing time is $T_{2,d} = 2T_1$, and the measurement-induced dephasing is $\gamma_\phi = 2\kappa = 0.8$ MHz. Thus the total dephasing is $\Gamma = \frac{1}{2T_1} + \gamma_\phi = 0.05$ MHz $+ 0.8$ MHz $= 0.85$ MHz. From this we can see that we expect the measurement-induced dephasing to dominate the dephasing of the qubit. A total runtime of $4\ \mu s$ ensures that the neural network sees at least 1 full Rabi cycle (appr. 3), aiding its estimation of Ω_R.

Given these parameters, it is possible to estimate a *Signal-to-Noise Ratio* (SNR) for the $\hat{\sigma}_z$-measurement[2], enabling us to examine how well the qubit state can be resolved from the noisy signal $J(t)$. Considering eq. 2.4, we know that $\langle\hat{\sigma}_z\rangle$ can maximally take on values ± 1. Following Naghiloo's ([24]) section 4.2.3, we can find the maximum spread in $J(t)$ across a time bin of size dt — denoted ΔJ.

[1] The Rabi frequency is especially important for the varying Rabi dataset.
[2] Similar calculations can be done for $\hat{\sigma}_x$ and $\hat{\sigma}_y$.

The probability distribution of J is Gaussian, with a mean determined by the value $2\sqrt{\kappa\eta}\langle\hat{\sigma}_z\rangle$ and the variance determined by the Wiener increment $\sigma^2 = 1/\Delta t$. For two maximally distinguishable outcomes $\langle\hat{\sigma}_z\rangle = \pm 1$, the separation between their distributions are $\Delta J = 4\sqrt{\kappa\eta}$, resulting in an SNR of

$$\text{SNR} = \left(\frac{\Delta J}{\sigma}\right)^2 = \frac{16\eta\kappa}{1/\Delta t} = 16\eta\kappa\Delta t = 0.02. \tag{3.3}$$

So within a single bin, the distributions pertaining to the two maximal outcomes are only separated by 0.14σ as seen by $\sqrt{0.02} \approx 0.14$. To obtain SNR $= 4$ — i.e., a 2σ separation between the two outcome probability distributions — the per-bin SNR needs to accumulate across ~ 200 bins, which is the onset of being able to distinguish the two maximal outcomes. This, we can also define as a measurement time (the time needed to measure to resolve the qubit states into a 2σ separation)([24] sec. 4.2.3):

$$\tau_m = \frac{1}{4\eta\kappa} \approx 2.08\,\mu s,$$

which is the measurement time needed to resolve the two maximal outcomes to a 2σ separation. It is worth stressing that this is for a qubit whose state does not change during the measurement — to see the Rabi dynamics, the measurement time needs to increases significantly. We will later employ this number in the discussion of whether it is possible to identify the Rabi frequency from a single trajectory.

3.2 Loss function

The loss function is the most crucial feature of the neural network model, since it determines the weight updates and thus the improvement in state prediction. We use a *Binary Cross-Entropy* loss function (see BCEloss in [6]). For a single trajectory prediction:

$$\mathcal{L}(P_{\text{end}}, y) = -[y \cdot \log(P_{\text{end}}) + (1-y) \cdot \log(1 - P_{\text{end}})] \tag{3.4}$$

It optimizes against the only measure available besides $J(t)$, namely the projective end measurement y. We optimize the entire model against this single measurement, since it is directly dependent on the quantum state. Since $y \in \{0, 1\}$, one might think that $P_{\text{end,NN}}$ ought also to be 0 or 1. But since y is determined by some Bernoulli-distribution, the best guess $P_{\text{end,NN}}$ turns out — across many trajectories — to be exactly the original true probability characterising the Bernoulli-distribution. If the models prediction differs from $P_{\text{end,true}}$, it evidently gets punished and thus changes the weights of the network. The choice of loss function is inspired by [7].

3.3 Model structure

The purpose of the model is to give the best prediction of the conditional state evolution $\rho_{true}(t)$, given only the measurement record $J(t)$ and the projective end measurement $y \in \{0, 1\}$ (and knowledge of which basis these come from). The idea is that you could train the model by simply doing measurements on an unknown system in the lab, and then the model will predict a physically valid $\rho_{NN}(t)$, which can inform quantum control on the system.

Figure 3.1 shows the model pipeline. The first step of the model is the LSTM network, which was described briefly in sec. 2.5. The defining structure of each LSTM cell can be seen in App. A. The first LSTM cell receives the first value in the measurement record, $J(t_0)$, along with the measurement direction $\hat{\sigma}_b$. The LSTM treats the input according to the equations in App. A. The dimensions of the weight matrices $\mathbf{W}$ are such that the hidden state and cell state are 64 dimensional. The LSTM cell passes the hidden state and cell

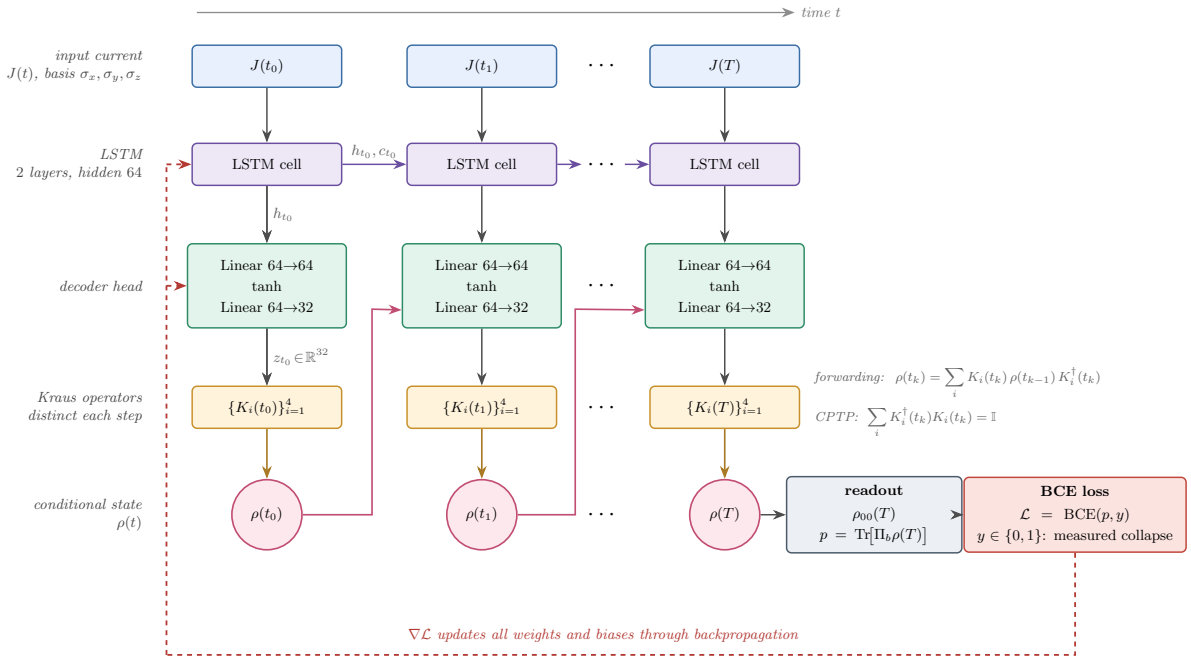

Figure 3.1: Schematic overview of the model structure. See main text for a walk-through. This schematic is realized by exact instruction prompts to Claude Opus 4.7, which converts instructions to TikZ-code (see Generative AI declaration).

state to the next cell, and also to a sequential linear neural network (green box in Fig. 3.1). The states of the LSTM are what allows for memory of earlier values of $J(t)$ through the network.

The job of the decoder head (the green boxes in Figure 3.1) is to convert the 64-dimensional array h_t that it receives from the LSTM into a 32-dimensional array that defines the four Kraus operators used to forward ρ_{NN}. Four 2×2 Kraus operators with complex entries result in 32 independent parameters (16 Re and 16 Im), which is the output dimension of the decoder head. There is a distinct decoder head at each time step for each of the three measurement directions.

Before the Kraus operators are applied to ρ_{NN}, they are modified such that the completeness relation is ensured (see sec. 3.4). Once forwarded, the process restarts with the next value in the measurement record $J(t_1)$ and so on for all values in $J(t)$ until the entire prediction record $\rho_{NN}(t)$ has been constructed.

From the last step $\rho_{NN}(T)$, the projective probability is determined. During the training of the model, this end probability is compared to the projective measurement $y \in \{0, 1\}$ through the loss function. The loss value is used to inform the weight and bias update through gradient descent, resulting in an improved performance.

The LSTM and decoder heads — and the weight update of these — are implemented using the library `Pytorch` [6].

3.4 Enforcing CPTP and forwarding

Before forwarding ρ_{NN} using the predicted $\hat{K}_j$, we ensure the CPTP-property[3] (see Theory) by enforcing $\sum_{j=0}^{3} \hat{K}_j^\dagger \hat{K}_j = \hat{I}$. As mentioned in sec. 2.4, a CPTP map ensures that ρ remains valid. Valid in this case means that the interpretation of ρ stays intact, namely that it is a description of a system with probabilities assigned to the possible states of that system. As the system is acted on by measurement, ρ_{NN} can be

[3] As will be established later, the network's Kraus operators are dependent on the same $\rho_{NN}(t_{k-1})$ that they act on, thereby — strictly speaking — moves the evolution into a regime where the term *CPTP maps* is not well-defined. See sec. 5.3.1 for elaboration.

updated step by step and one needs to ensure that the interpretation of ρ_{NN} stays valid. A CPTP map retains the physicality of ρ_{NN} while accurately updating ρ_{NN} according to what affects the system. Without such a constraint, the model would be free to output almost anything, including a ρ_{NN} that does not fit such an interpretation. Thus, whatever the model proposes, using CPTP, ρ_{NN} is still a legitimate state a real system could occupy and so remains physically interpretable.

Practically (i.e., in the code), the unity criterium — the trace-preservation — can be restated as a single matrix equation by stacking the Kraus operators in a new (8×2) matrix M:

$$\sum_{j=0}^{3} \hat{K}_j^\dagger \hat{K}_j = \begin{bmatrix} K_0^\dagger & K_1^\dagger & K_2^\dagger & K_3^\dagger \end{bmatrix} \begin{bmatrix} K_0 \\ K_1 \\ K_2 \\ K_3 \end{bmatrix} = M^\dagger M = \begin{bmatrix} \langle \mathbf{u}, \mathbf{u} \rangle & \langle \mathbf{u}, \mathbf{v} \rangle \\ \langle \mathbf{v}, \mathbf{u} \rangle & \langle \mathbf{v}, \mathbf{v} \rangle \end{bmatrix} = \hat{I} \tag{3.5}$$

Here, $\mathbf{u}$ and $\mathbf{v}$ are each of the two columns in M, respectively. The criteria can therefore be reformulated as $||\mathbf{u}|| = ||\mathbf{v}|| = 1$ and $\langle \mathbf{u}, \mathbf{v} \rangle = 0$. The `PyTorch QR` algorithm changes the values in $\mathbf{u}$ and $\mathbf{v}$ (and thus also $\hat{K}_j$) in order to meet these criteria.

The Kraus operators — for each measurement direction — then forward ρ using $\rho(t_{k-1})$ (we provide the initial state to the first forwarding step), in accordance with eq. 2.11.

To ensure that the predicted ρ does not deviate much from the initial state around $t = 0$[4], we bias the first Kraus operator to $\hat{K}_0 = \hat{I}$, thus — from the completeness relation — $\hat{K}_1,\ \hat{K}_2,\ \hat{K}_3 = \hat{0}$.

3.5 Training the model

Training the model means determining all the weights and biases in a way that minimizes the loss for the training data and an unknown validation set[5]. The training process is characterised by *hyperparameters* (see Table 3.2), which determines how the training dataset is processed by the model. The training was done on $N = 30000$ $J(t)$ records with an equal distribution in measurement direction.

`Hyperparameter name`	**Value**	**Description**
Batch size	64	Number of $J(t)$ records the model processes before doing a weight update
Epochs	20	Number of times the model goes through the entire training dataset of N $J(t)$ records
Weight decay	10^{-4}	Additional loss that penalizes larger weights[a]
Learning rate interval	$10^{-3} \rightarrow 10^{-5}$	Coefficient that determines the size of the weight update in $\nabla\mathcal{L}$. Goes through interval with *Cosine Annealing Scheduler*
LSTM hidden layers	$\#2, 64\,\mathrm{dim}$	Determines the network structure that is repeated for each LSTM cell.

[a] This usually counteracts over-fitting by discouraging the model from relying too heavily on any individual feature, hence typically improving its generalisation [16].

Table 3.2: Hyperparameters used in the training of the model.

We use the AdamW weight optimizing algorithm [3] together with a cosine annealing learning rate scheduler [4]. This means that the learning rate gradually decreases from first to last epoch, allowing a better fine-tuning of the weights near the end of the training. Figure 3.2 shows both the evolution of the losses and the learning rate through the epochs.

[4] Large deviations from the early ρ_{true} constituted a major problem before the introduction of the bias term, motivating the implementation.

[5] Unknown means that the back-propagation (i.e., the so called gradient descent) is disabled when applied to validation data, deactivating the updating of weights

If the model were just guessing P_{end} randomly (uniformly), it would result in $\langle P_{\text{end,NN}} \rangle = 0.5$ leading to $\mathcal{L}(0.5, 0) = \mathcal{L}(0.5, 1) = \log(2)$ (cf., eq. 3.4). Likewise, even if the model were to predict $P_{\text{end,NN}}$ in perfect correspondence with $\rho_{\text{true}}(T)$, it would still be wrong sometimes ($\frac{1}{N}\sum_i^N \mathcal{L}(\rho_i^{\text{true}}(T), y_i) > 0$) due to the Bernoulli distribution that y is drawn from. These two limits are drawn on Fig. 3.2, and it is observed that the model trained on data with a fixed Ω_R gets close to the irreducible value.

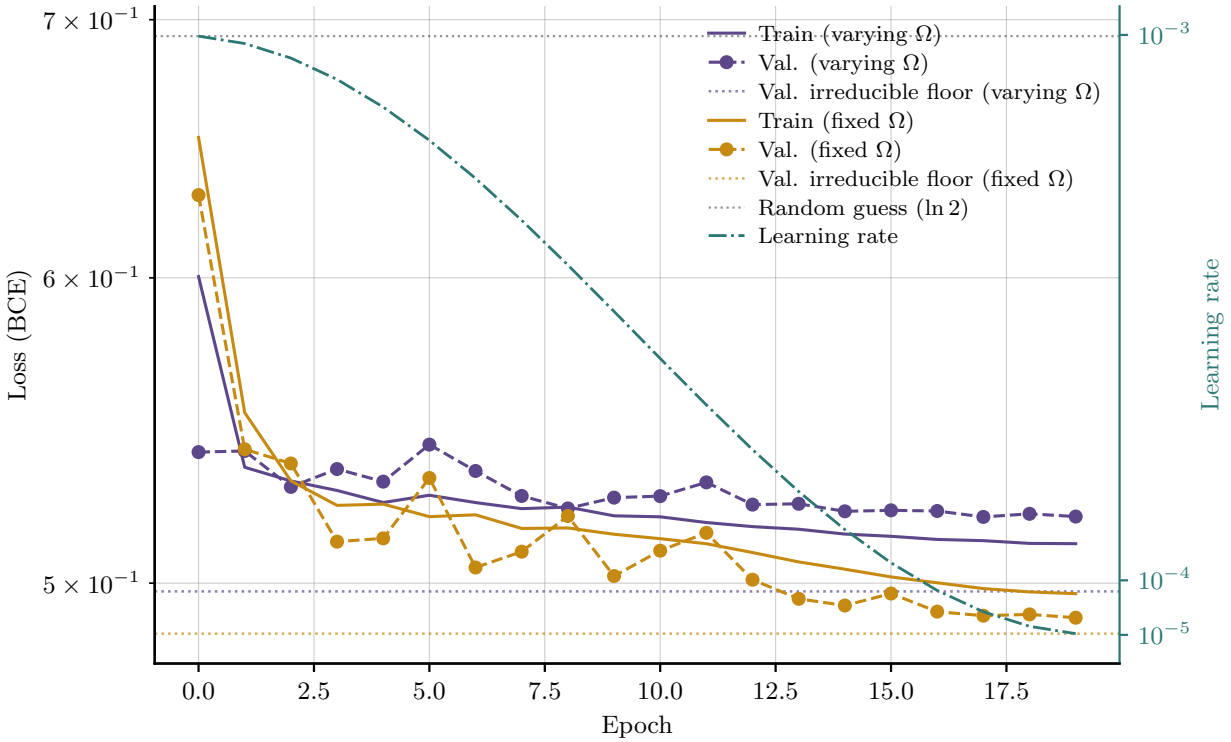

Figure 3.2: Mean loss per trajectory for every epoch of the training, both training (solid) and validation (dashed). The training on trajectories with different (*varying*) Rabi frequency are plotted with purple curves, whereas the yellow curves are for the training data with fixed Rabi frequency. The horizontal dotted lines, purple and yellow, mark the irreducible BCE-floors computed with eq. 3.4 for the varying and fixed Rabi validation data respectively. The weights slowly converge towards a stable minimum. The learning rate (dot–dashed green line) decays as training progresses allowing for a better finetuning of the weights.

The model was trained on a MacBook Pro 2025 M5, resulting in a runtime of approximately 2 hours.

Model Performance and Evaluation

4

We will now proceed with the trained model and apply it to 10,000 new trajectories, per $\hat{\sigma}_{b_i}$[1], in order to evaluate its performance. $\rho_{\text{true}}(t)$ is the QuTiP-generated trajectory obtained in the simulation alongside $J(t)$, and is used as the true state reference. $\rho_{\text{NN}}(t)$ is the model's predicted density matrix conditioned on $J(t)$.

4.1 Single trajectory reconstruction

Figure 4.1 shows the model prediction for a specific $\hat{\sigma}_z$ trajectory with fixed Ω_R. There is good qualitative agreement between the prediction and the true state for ρ_{00}[2], which suggests that the model is successful in conditioning its prediciton on $J(t)$. The model also captures the dynamics of $\text{Im}(\rho_{01})$ — the y-component of the Bloch vector — although no direct information connected to off-diagonal elements exists in the $J(t)$ signal. This is likely due to a combination of the constraint that the full ρ_{NN} must be valid and that some transfer of information from the $\hat{\sigma}_y$ training through the shared LSTM (more about this phenomenon in sec. 4.4), which effectively seems to encode the Hamiltonian rotation. The immediate poor construction of $\text{Re}\,\rho_{01}$ is presumably the adverse effect of a purity bias, which will be examined more closely in the section just mentioned.

Examples of single trajectory reconstructions in $\hat{\sigma}_x$ and $\hat{\sigma}_y$ are provided in Appendix D.

4.2 Fidelity between ρ_{NN} and ρ_{true}

To compare ρ_{NN} and ρ_{true} quantitatively across all trajectories, we employ the measure of fidelity as laid out in [21]:

$$\text{F}(\rho_{\text{NN}}, \rho_{\text{true}}) = \text{Tr}\sqrt{\sqrt{\rho_{\text{NN}}}\rho_{\text{true}}\sqrt{\rho_{\text{NN}}}} \tag{4.1}$$

Qualitatively, fidelity is a measure (a number between 0 and 1) of the similarity between two quantum states, i.e. full density matrices. It is therefore also a favorable single statistic evaluation of the model's performance. Fig. 4.2 shows the averaged fidelity across time for all three measurement directions, for both the varying and fixed Rabi frequency.

The main takeaway from the fidelity analysis is that it confirms our presumption that the model performs best in the fixed regime. However, it is also suggestive of the average dynamics that a qubit experiences. For example, the graph for fixed $\hat{\sigma}_x$ oscillates towards a higher fidelity. The $\hat{\sigma}_x$ operator does not receive information about the Hamiltonian dynamics in the yz-plane of the Bloch-sphere, and the model is therefore struggling in the beginning, since the true dynamics are in exactly this plane. The measurement gradually collapses the state towards one of the σ_x eigenstates and in the process makes the Hamiltonian influence on ρ_{00} and $\text{Im}(\rho_{01})$ less and less by rotating the Bloch vector away from the yz-plane, thus improving the prediction. See section 4.4 on ensemble averages for a further discussion.

4.3 Varying Rabi trajectories

Fig. 4.3 shows three specific trajectories for the model trained on varying Rabi frequencies. It is — assessed on the basis of these representatives — visually clear that the performance is worse compared to the model trained only on a fixed Ω_R (Fig. 4.1). The fit values of Ω_R (displayed above each trajectory in Fig. 4.3) shows that there is a clear tendency for the model to chose the frequency that is close to the mean of the

[1] In this context, $\hat{\sigma}_x$, $\hat{\sigma}_y$ and $\hat{\sigma}_z$ are short for the associated dataset monitored and projected in the respective basis.
[2] For brevity, we omit (t) from the elements of ρ, as it is implicitly understood that the full time series is considered.

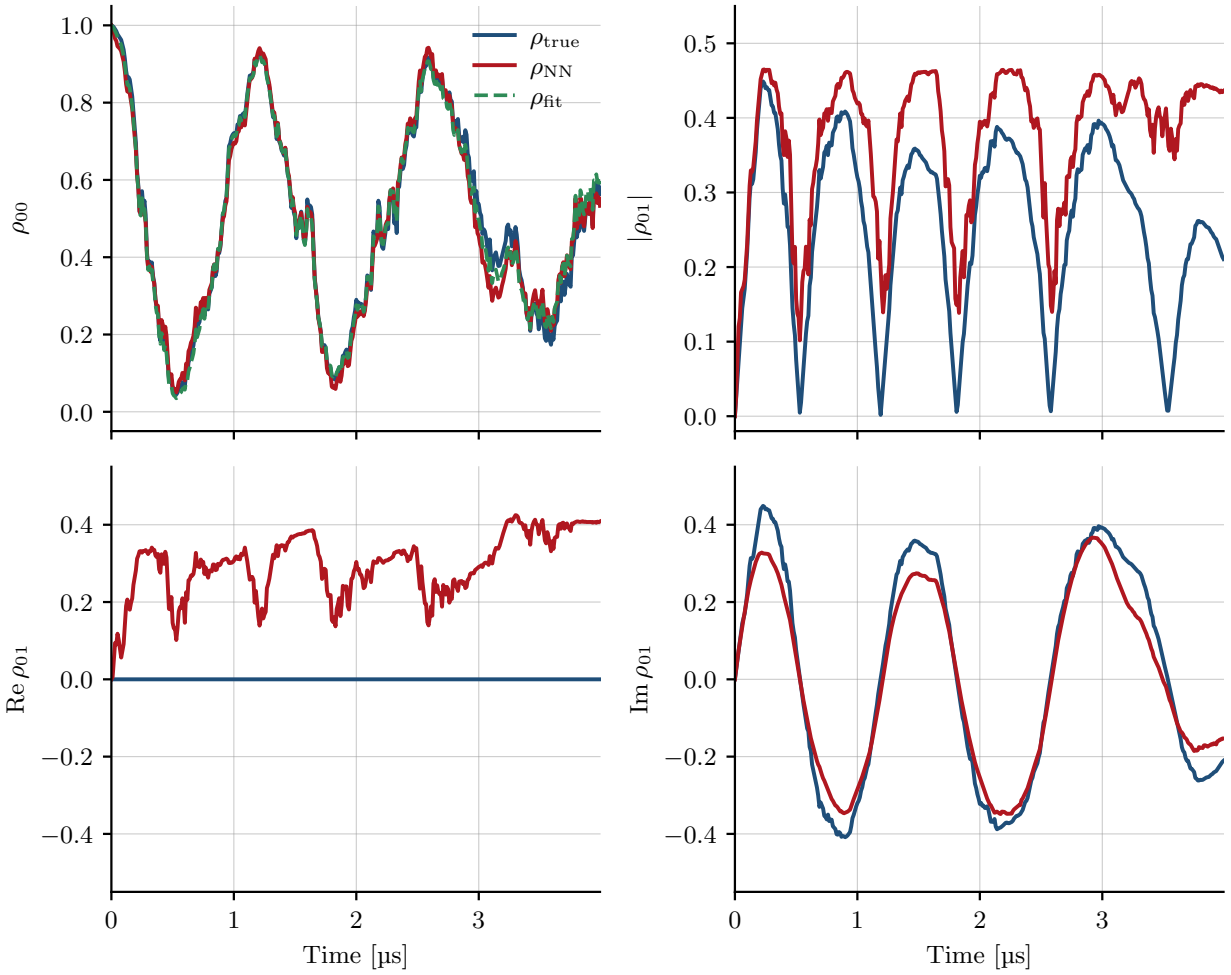

Figure 4.1: Reconstruction of the density matrix $\rho(t)$ by the model for a specific realization, trained and evaluated on z-basis measurement records at a single fixed Rabi frequency, and informed by a final projective measurement in that same basis. The panels show the population ρ_{00}, the coherence magnitude $|\rho_{01}|$, and its real and imaginary parts $\text{Re}(\rho_{01})$ and $\text{Im}(\rho_{01})$. Blue: ground truth ρ_{true} from `smesolve`; red: model reconstruction ρ_{NN}; green dashed: ρ_{fit} — an SME fit to the reconstructed population ρ_{00}^{NN}.

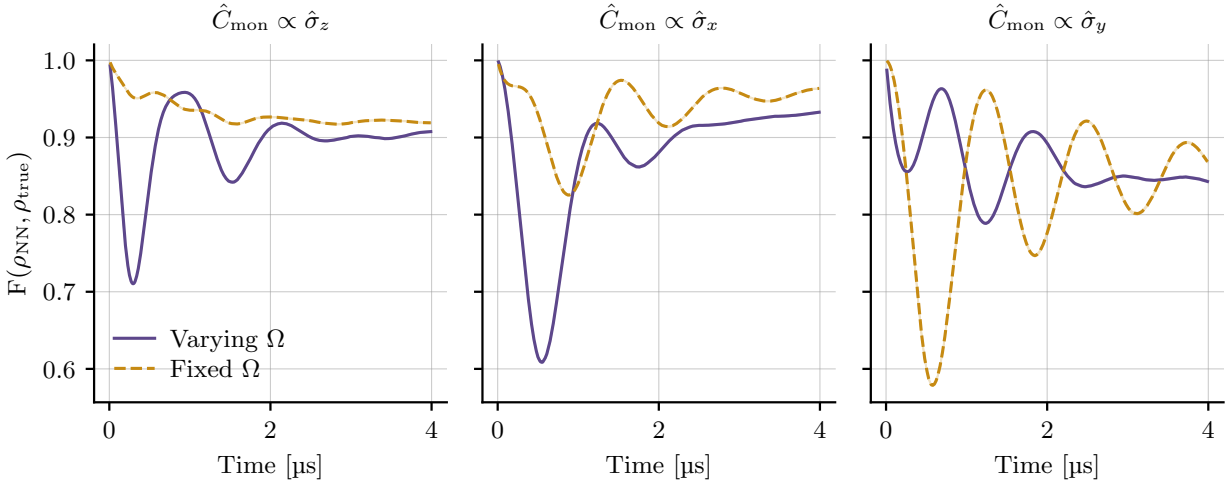

Figure 4.2: Fidelity (definition from [21]) between ρ_{NN} and ρ_{true} shown as a function of time. The purple curves correspond to the model trained and evaluated on varying Rabi frequencies, while the yellow curves correspond to the model trained and evaluated on a single fixed Rabi frequency. For each of the curves there is a shaded region — too small to be seen due to the $N = 10000$ trajectories — showing 68% confidence-interval from bootstrapping.

sampled distribution ($\Omega_R \approx 5$ MHz). It tells us that the model is not capable of correctly predicting this variation in frequency from sample to sample (by employing the LSTM's memory capabilities), and instead ends up choosing a single (mean) frequency that minimises the loss in average.

Since the varying model must recognize Ω_{R_i} on a sample basis, it must extract this information from $J(t)$. This is not strictly required in the fixed frequency case, since it is just a constant across all trajectories, which it could potentially just encode in an (unconditional) bias. The varying model does not have this privilege, and it turns out to be too difficult a task. It might be due to the very low SNR in $J(t)$, cf. section 3.1.2. Scaling of the training; increasing the duration of each record, as to show more oscillations; and changing the model hyper-parameters are all potential remedies. However, we did not observe a noticeable improvement implementing the solutions highlighted by the above diagnostics[3], which hints at an issue of higher difficulty, exceeding the capabilities of this bachelor project.

4.3.1 Parameter estimation from ρ_{NN}

Assuming that the model has predicted a physically valid ρ (Fig. 9.1 in App. B) — i.e., positive semi-definite and unit trace — we try to extract the physical parameters $\vec{\theta} = (\Omega_R,\, \kappa,\, \eta,\, \gamma_d)$ that would have given rise to such a trajectory.

Within $J(t)$ lies information about the observable $\hat{\sigma}_{b_i}$, thus providing information about one component of the qubit's Bloch vector, or equivalently one element of ρ (see section 2.1.2). We observe that the model performs best on data obtained from measuring with $\hat{\sigma}_z$, and therefore we use ρ_{NN} obtained from $\hat{\sigma}_z$ measurements to extract the physical parameters. As described in section 2.3, the Stochastic Master Equation (SME) allows one to compute the increment $d\rho$ conditioned on a continuous measurement record with knowledge of which operators act on the system. At each increment, the SME computes two contributions to $d\rho$: a deterministic (unconditional) part according to the Hamiltonian and Lindblad operators and a stochastic (conditional) part driven by the measurement record ([24] secs. 4.3.1-2). If we specify the system operators and provide the SME with $J(t)$, a full ρ_{fit} can be constructed by numerically integrating, with $\vec{\theta} = (\Omega_R,\, \kappa,\, \eta,\, \gamma_d)$ as free parameters, which we can then compare with the neural network's prediction ρ^{NN}. Using a loss function (see eq. 4.2) that penalises the fit for creating a ρ_{fit} whose population ρ_{00}^{fit} does not match ρ_{00}^{NN}, it is possible to tune the parameters of the SME to create a ρ_{fit} that most closely resembles ρ_{NN}. Since we measure with $\hat{\sigma}_z$, this informs us directly about ρ_{00}^{true} and the dynamics (determined by $\vec{\theta}$) are imprinted on how ρ_{00} evolves, hence matching ρ_{00} is sufficient to obtain values for $\vec{\theta}$ and the rest of ρ^{fit}. Using this method, we can also see to which degree Ω_R can be inferred per-trajectory. We specify the fit with the same operators as described in sec. 3.1.1, and the fit minimizes the loss function

$$\mathcal{L}_{\text{fit}} = \frac{1}{N+1} \sum_{k=0}^{N} |\rho_{00}^{\text{fit}}(t_k; \vec{\theta}) - \rho_{00}^{\text{NN}}(t_k)|^2, \tag{4.2}$$

where N is the number of steps in the measurement record. $\mathcal{L}_{\text{fit}}$ is minimized using the 'L-BFGS-B'-optimization algorithm [29] initialized at the true values of $\vec{\theta}$ (see table 3.1), giving us the parameters of the SME that constructs a ρ_{00}^{fit} closest to ρ_{00}^{NN}. This fit is done on a subset of all trajectories. The resulting residuals of parameters compared to the true parameters are shown in Fig. 4.4. Three different fits are shown in Fig. 4.3. Although the fits are broadly acceptable, they are evidently not perfect — consequently, the estimated parameters and their resulting interpretations must naturally be treated with caution[4].

The most apparent conclusion to draw from Fig. 4.4 is that the varying Ω_R poses a difficulty to the model. Examining the top left plot, it is clear that for low Ω_R^{true}, the model overestimates Ω_R, and for high Ω_R^{true}, it underestimates, indicating that the model is regressing to the mean.

[3] These results would preferably have been presented in the project — but in the lack of time, this must be left as an unsubstantiated claim.

[4] Optimally, errors on the estimated parameters should have been extracted from the fit. In the absence of these, it has nevertheless been attempted to analyse the model's ability to reproduce the physical quantities, assuming that the fit is approximately ideal.

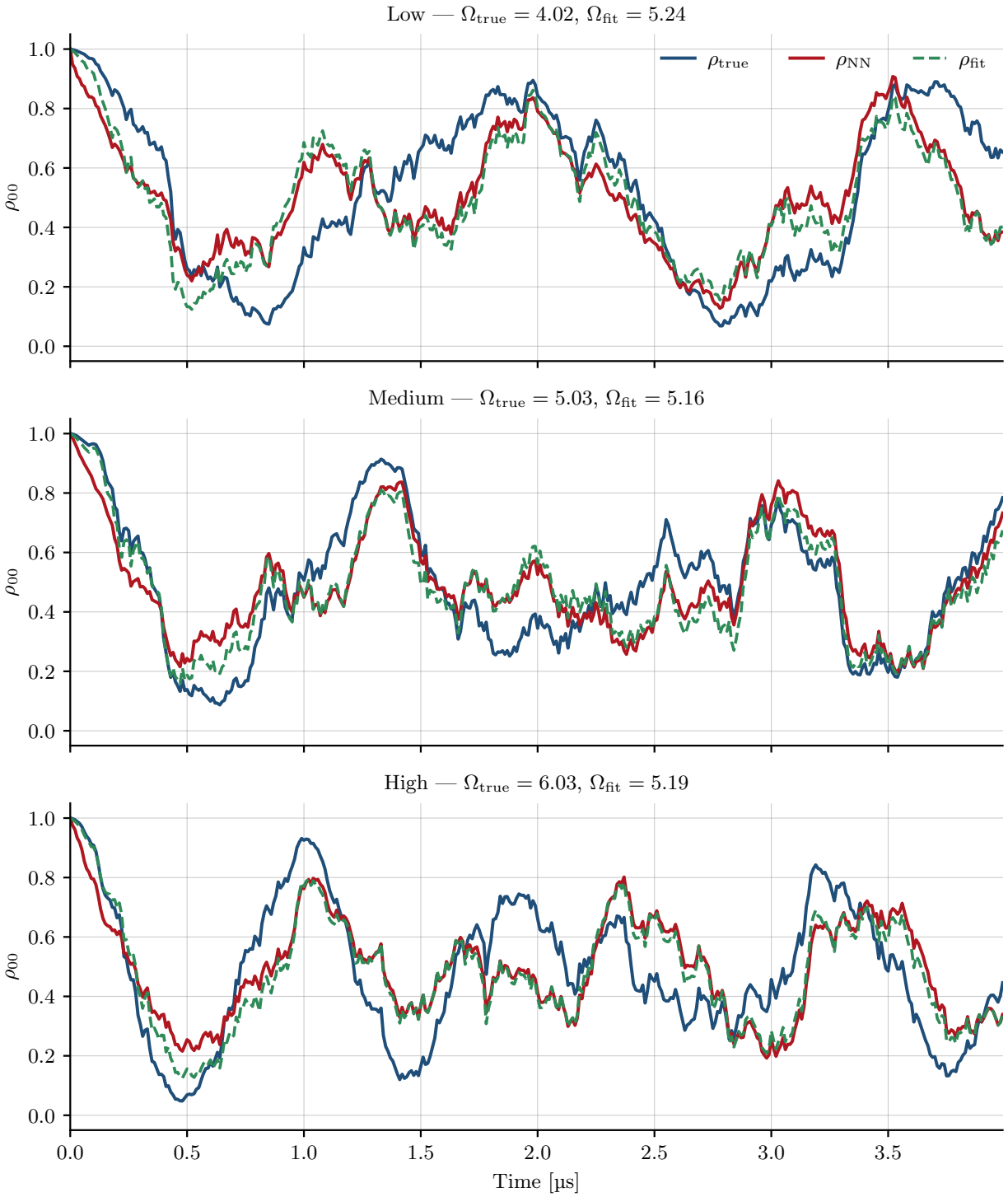

Figure 4.3: Reconstruction of the population $\rho_{00}(t)$ by the model, trained and evaluated on z-basis measurement records for three representative Rabi frequencies (low, intermediate, and high), sampled from a uniform distribution centered at $\Omega = 2\pi \times 0.8$ with relative spread 0.2. Blue shows the ground truth ρ_{true} from `smesolve`; red shows the model prediction ρ_{NN}; and green dashed shows ρ_{fit}, obtained via a least-squares fit of SME parameters using `scipy.optimize.minimize` with the L-BFGS-B algorithm. The panel subtitles list the corresponding Rabi frequency and fitted population estimate.

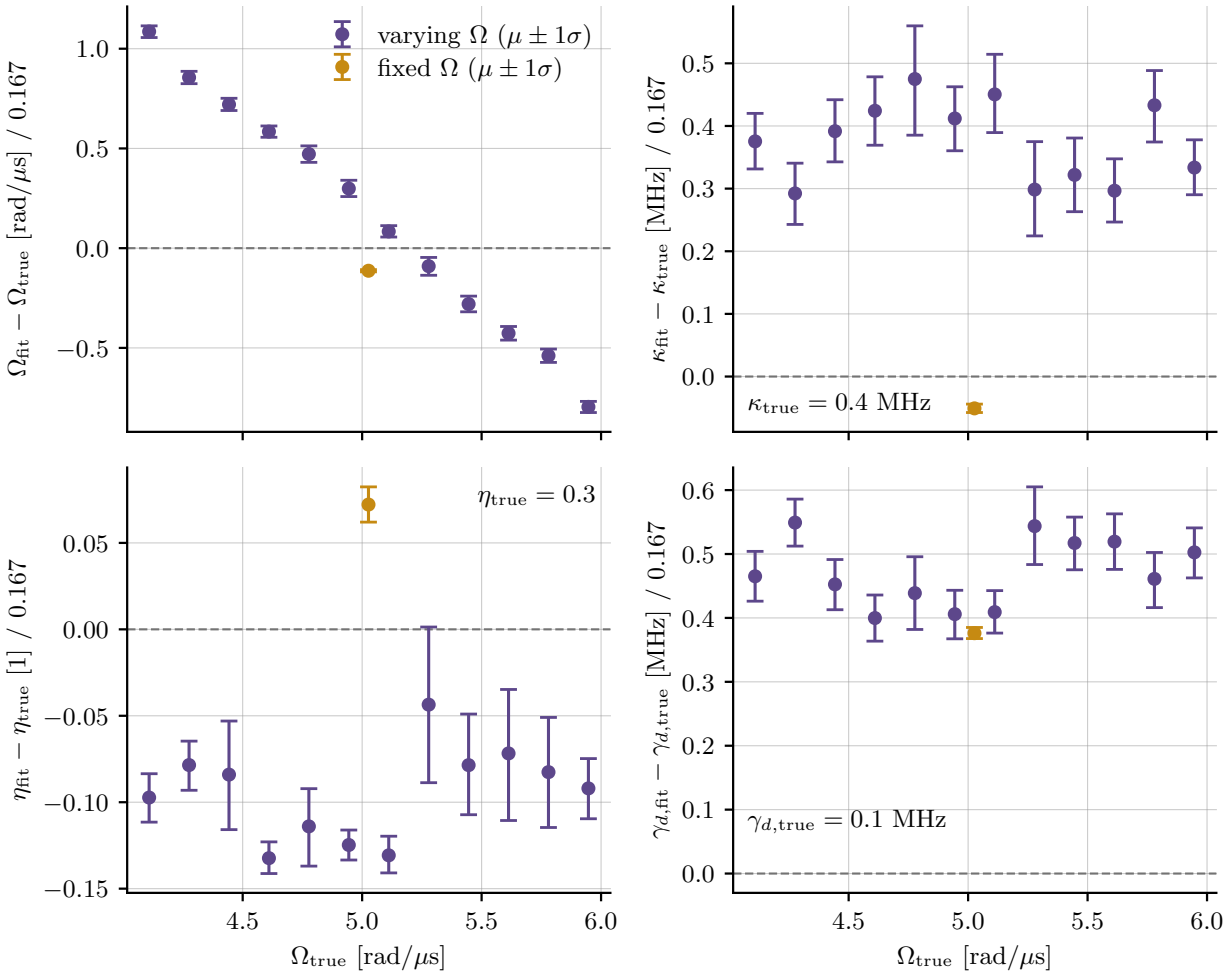

Figure 4.4: Residuals of estimated parameters (Ω_R, κ, η, γ_d) as function of the true Ω_R in both the fixed (orange) and varying (purple) Ω_R cases obtained from fitting the SME to ρ_{00}^{NN} with $\hat{C}_{\text{mon}} \propto \hat{\sigma}_z$ for every 30th trajectory in the evaluation data set (i.e., ~ 300 total). In the varying Ω_R-case, the frequencies are drawn from a uniform distribution per-trajectory (see table 3.1).

Since the model is not successful in estimating Ω_{R_i} for individual measurement records, it points to the fact that the oscillatory dynamics we observe is not conditioned on $J(t)$. Cf., the equations defining the LSTM structure in App. A, we see that there are two influential factors in determining the output hidden state: the weights $\mathbf{W}$, which act on the input $J(t)$, and the biases $\vec{b}$, which are independent.

As established previously (sec. 3.1.2), the measurement time needed to resolve the qubit state to Naghiloo's standard of SNR $= \sigma^2 = 4$ is $\tau_m = 2.08\,\mu s$. The chosen Ω_{R_i} results in a varying Rabi period of $T_{\text{R, var}} \in [1.04\,\mu s,\ 1.56\,\mu s]$ and the fixed Ω_R a period of $T_{\text{R, fix}} = 1.25\,\mu s$. In all cases, the Rabi period is shorter than τ_m, so the qubit completes more than a full oscillation before the measurement can resolve its state. Measurement progresses for $T = 4\,\mu s$ meaning that measurement is effectively exposed to Ω_R for twice the measurement time or ~ 3 Rabi cycles, and the question is whether Ω_R can be estimated from so few cycles.

Since the model has not been able to resolve the oscillations from the signal, the most likely conclusion is that the network has learned to produce oscillating trajectories through the loss landscape when optimizing the bias vector. This would explain why every trajectory oscillates with nearly the same Ω_R, since the bias vector is independent on the input current, $J(t)$. This aligns well with our experience that more training data did not result in better predictions of Ω_{R_i}. It also means that the model has the potential to make better predictions given a higher SNR, so the oscillatory dynamics in $J(t)$ would be resolvable.

Moving on to the other subplots in Fig. 4.4, we see that the fit under- or overestimates the other parameters greatly in comparison to when the Rabi frequency is fixed. Since the model reproduces dynamics around the mean Ω_R rather than the trajectory's true value, no single set of parameter $\vec{\theta}$ reproduces the trajectory exactly. When trying to match ρ_{00}^{NN}, the fit has to change κ, η, γ_d away from their initial (true) values to compensate for the mismatched oscillations.

κ is systematically overestimated, suggesting a higher measurement strength, hence also more measurement-induced dephasing. With the efficiency η estimated lower than the true value, the effective $\hat{C}_{\text{loss}}$ decoheres the off-diagonal elements with larger strength than in reality and less information reaches the detector. With γ_d estimated higher than in reality, the state relaxes faster, experiences more dephasing, and oscillations in ρ_{00} are damped (for elaborate explanations, go back to subsec. 2.1.1). All of these parameters indicate that the Bloch vector is shortened continuously, thus dampening the oscillations in ρ_{00}^{NN} faster than ρ_{00}^{true}. A possible reason for the model to produce trajectories that — when fitted with an SME trajectory — produce parameters corresponding to larger decoherence is its loss function. In general (supported by examining Fig. 11.1, left plot, in App. D), we see that the model's predicted distribution of outcomes has a lower variance — thus a sharper distribution — around the mean value of $P_{\text{end}} \simeq 1/2$, when compared to the true distribution. Since the predicted outcome cannot match each true outcome, the most viable strategy to minimize the BCE is by predicting probabilities around the mean value of P_{end} rather than to commit to extreme values that it does not consistently estimate. To realize such distribution of probabilities, the state is required to have a low outcome variance around the expectation value of the measurement operator on the measurement axis, which it can do by lowering the amplitude of oscillations. A possible way for an SME fit to account for lowered amplitude of oscillations is through decoherence. This might be why κ and γ_d are overestimated and η underestimated.

4.4 Ensemble-averaged reconstructions and purity

While the per-trajectory reconstructions are the actual purpose of the model, it is informative to ask what the typical reconstruction looks like once the record-specific fluctuations are removed. Averaging the conditioned states over all evaluation trajectories yields an effectively unconditioned state: The stochastic Wiener increments have zero mean, they average out at every time step, leaving only the deterministic part of the dynamics. The average of the true conditioned states must therefore reproduce the unconditional Lindblad evolution at the true parameters. Fig. 4.5 confirms this, since the blue ($\overline{\rho}_{\text{true}}$) and dashed green (QUTIP Lindblad) curves are indistinguishable.

Comparing the averaged predicted states $\overline{\rho}_{\text{NN}}$ (red) to the Lindblad reference thus isolates the deterministic, ensemble-level physics the network has internalised, independently of any particular noise realisation. A systematic deviation of $\overline{\rho}_{\text{NN}}$ from Lindblad therefore highlights some bias in the network.

It is useful to emphasise which elements of ρ receive a direct training signal (the density matrix is written in the z-basis throughout Fig. 4.5). The loss function depends only on P_{end}, the projective measurement probability in the monitored basis — the diagonal element of ρ when written in the measurement basis. For the three rows, this is $\text{Re}\,\rho_{01}$ ($\hat{\sigma}_x$), $\text{Im}\,\rho_{01}$ ($\hat{\sigma}_y$) and ρ_{00} ($\hat{\sigma}_z$), respectively (the diagonal panels of the grid). Every other element in the subplot is constrained only indirectly, through the following two mechanisms:

1. **The CPTP structure of the Kraus map.** A linear algebra theorem states that the determinant of a complex, square matrix A is equal to the product of its eigenvalues ($\det(A) = \Pi_i \lambda_i$) [32]. Using the positive semi-definite property of ρ (that it has positive eigenvalues):

$$\det(\rho) = \rho_{00}\rho_{11} - \rho_{01}\rho_{10} = \lambda_1\lambda_2 \geq 0 \tag{4.3}$$

$$\Rightarrow \rho_{00}\rho_{11} \geq |\rho_{01}|^2 \tag{4.4}$$

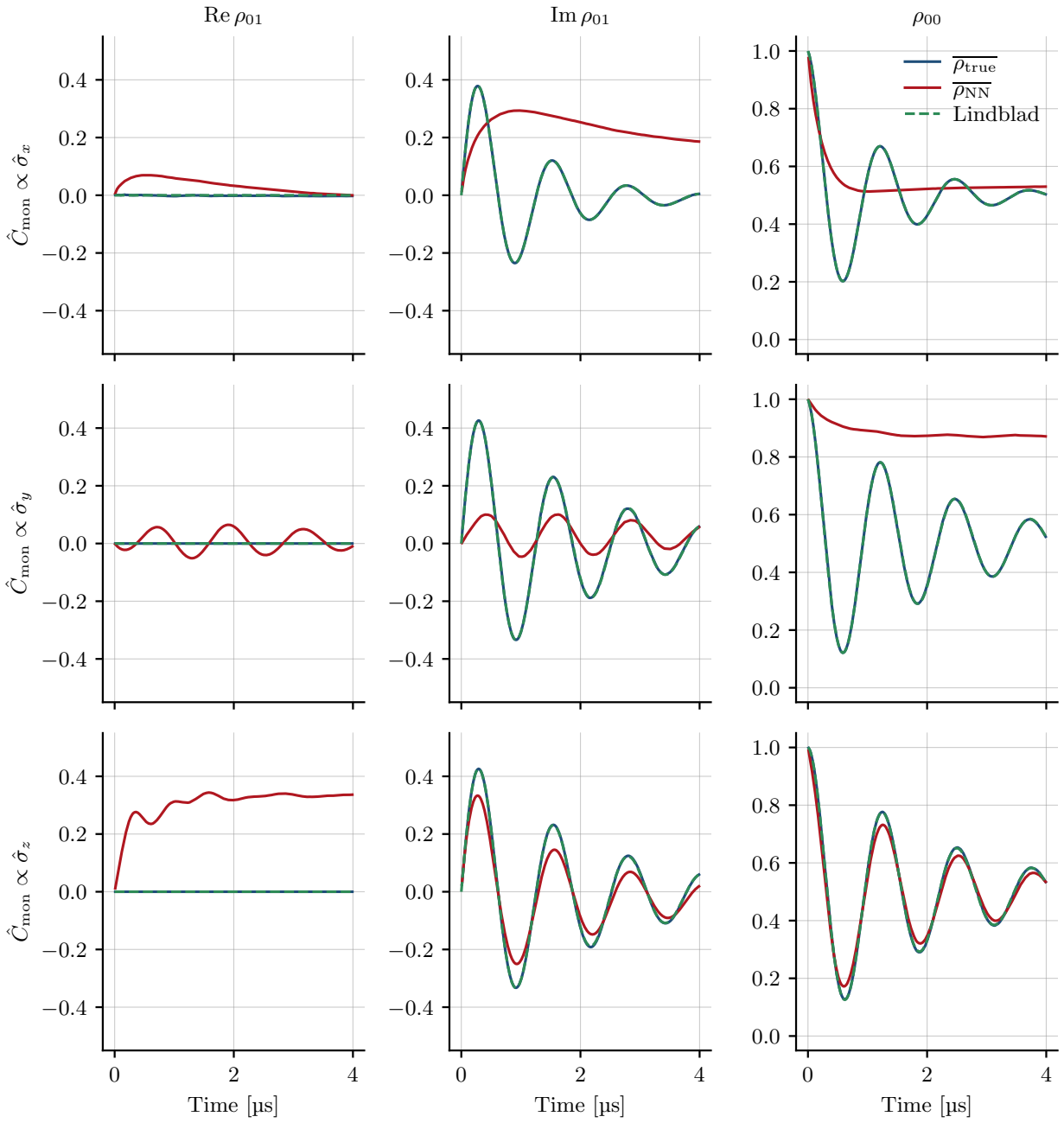

Figure 4.5: Ensemble-averaged reconstructions of ρ for a single fixed Rabi frequency shared across all trajectories. Rows correspond to the monitored operator $\hat{C}_{\text{mon}} \propto \hat{\sigma}_x, \hat{\sigma}_y, \hat{\sigma}_z$; columns show the population ρ_{00} and the real and imaginary parts of the coherence ρ_{01} (in the z-basis). For each basis the 10,000 conditioned realisations are averaged into an effectively unconditioned state: the network average $\overline{\rho}_{\text{NN}}$ (red), the true-state average $\overline{\rho}_{\text{true}}$ (blue), and a numerical (QuTip) Lindblad reference at the true parameters (dashed green).

Thus the off-diagonal elements are constrained by the diagonal elements, even when they do not influence P_{end}.

2. **The shared LSTM backbone:** Because the decoder heads are independent across measurement directions, the only route by which information learned in one basis can inform another is the shared LSTM weights, allowing for a more informed reconstruction of the elements orthogonal to the monitored axis.

There are two additional factors that influence the reconstructions. (1) Firstly, before the network makes its first weight update, we initialise the Kraus operators as one identity and three zero matrices. The starting point is therefore unitary dynamics (that are purity-preserving, cf. chap. 2). Since the initial state is always prepared pure ($|\psi_0\rangle = |g\rangle$), the network will keep it pure as long as no training (through gradients) changes the Kraus matrices. This is especially impactful for the coherence elements (relative to the monitored axis), since information about these are not contained in the current $J(t)$ directly. (2) Secondly, the BCE loss acts on the diagonal elements and is by construction reluctant to commit to an extreme P_{end} near 0 or 1: such a confident prediction is penalised heavily unless the network consistently estimates $\rho_{00}(t = T)$ near correctly.

The strongest reconstruction is notably ρ_{00} in the $\hat{\sigma}_z$ basis (bottom right). This is due to this element being the only one which is both supervised and has the initial configuration as an eigenstate.

The supervised $\operatorname{Im}\rho_{01}$ in the $\hat{\sigma}_y$ row (middle middle) does worse: it oscillates with the correct phase but too small an amplitude. $|g\rangle$ is a superposition in the $\hat{\sigma}_y$ basis, yielding a minimal SNR (see sec. 3.1.2) due to $\langle\sigma_y\rangle_{t=0} = 0$[5]. Thus, the model is rewarded for keeping its prediction, and hence the amplitude, cautious according to the BCE loss that punishes extreme P_{end} predictions more.

The $\hat{\sigma}_x$ row is the most diagnostic of the three (top row). Since $[\hat{H}, \hat{\sigma}_x] = 0$, the monitored observable is conserved by the drive, so the Rabi dynamics is absent from the record. This makes it a clean test of what the network can infer drive-independently. First, the reconstructed population (top right) decays monotonically, without the Rabi oscillation of the true state. The record carries no information about the oscillating z-population, so the network cannot reconstruct an oscillation it never sees. The true ρ_{00} settles near $\frac{1}{2}$, which means that the supervised $\operatorname{Re}\rho_{01}$ can approach $\pm\frac{1}{2}$ (mean 0), while staying within the Bloch sphere.

The remaining panels expose the purity bias in the elements that receive no direct signal. The clearest case is ρ_{00} for the $\hat{\sigma}_y$ measurements (middle right), where $\overline{\rho}^{00}_{\text{NN}}$ stays close to its initial value instead of decaying towards $\frac{1}{2}$. The network has no incentive to change the purity (priored from the initial Kraus operators), and therefore the population stays nearly pure. The same pattern seems to emerge in the rise of $\operatorname{Re}\rho_{01}$ to ≈ 0.33 in the $\hat{\sigma}_z$ row (bottom left) and the elevated $\operatorname{Im}\rho_{01}$ in the $\hat{\sigma}_x$ row (top middle). It could, as well, explain the anomalous and phase-shifted (in relation to $\operatorname{Im}\rho_{01}$ (middle middle)) oscillation in $\operatorname{Re}\rho_{01}$ (left middle) as an effort to maintain a consistently high purity in response to the concurrent oscillation in $\operatorname{Im}\rho_{01}$ (middle middle).

The purity bias is further illustrated with Fig. 4.6. An eigenstate of the monitoring basis will have zero off-diagonal elements (when written in that basis). In this case, no dephasing can happen, and purity loss will be minimal, since mixing can only be generated through relaxation and measurement inefficiency. By the same logic, a pure state orthogonal to the measurement axis will be highly susceptible to dephasing (and thus purity loss) due to its large coherence. Because $|g\rangle$ is a $\hat{\sigma}_z$ eigenstate, it carries maximal coherence with respect to both the $\hat{\sigma}_x$ and $\hat{\sigma}_y$ axes (it starts in a pure state, an equal superposition along these axes). For this reason, the true purities (blue curves for $\hat{\sigma}_x$ and $\hat{\sigma}_y$) both dephases strongly at early times, and the two purities fall steeply and together. For $\hat{\sigma}_z$, the state starts in an eigenstate of the monitored basis, and therefore the early dephasing is weak, and the purity falls more slowly. The same rule drives the oscillations for true $\hat{\sigma}_z$ and $\hat{\sigma}_y$: for $\hat{\sigma}_z$, the purity is roughly constant when the state is diagonal in the z-basis; for

[5] This is opposed to ρ_{00} for $\hat{\sigma}_z$, which — by the same logic — has a large initial SNR due to maximal $\langle\sigma_z\rangle$ $(= 1)$.

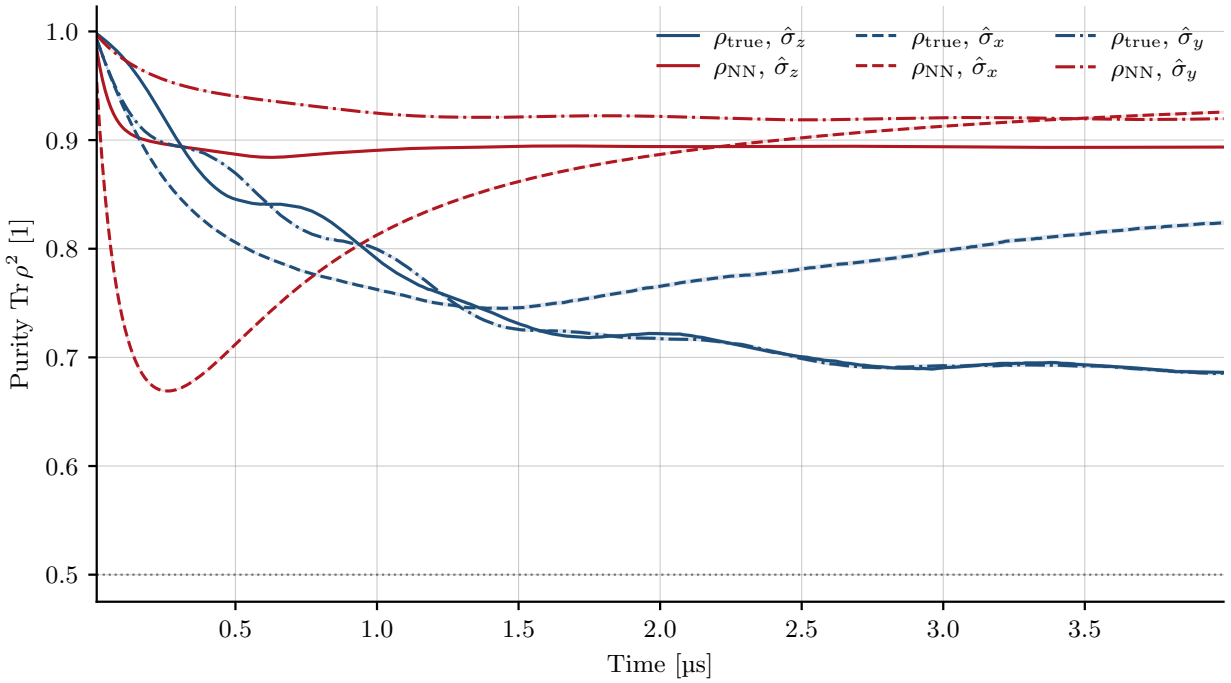

Figure 4.6: Purity $\mathrm{Tr}\,\rho^2$ averaged over the 10,000 conditioned trajectories of the fixed-Rabi datasets for each monitored operator. The quantity shown is the average of the per-trajectory purities. The true conditional states (blue) are compared with the network reconstruction (red), with line style encoding the measurement basis ($\hat{\sigma}_z$ solid, $\hat{\sigma}_x$ dashed, $\hat{\sigma}_y$ dash-dotted). The dotted line marks the maximally mixed value $\mathrm{Tr}\,\rho^2 = \frac{1}{2}$. An uncertainty band (computed with bootstrapping) — representing 68% CI deviation on the mean — is plotted as well for every curve, although it is so thin, it cannot be seen.

$\hat{\sigma}_y$, it is constant when the state is diagonal in the y-basis. Since the drive sweeps the state through the yz-plane, these plateaus fall half a Rabi period apart — so the two oscillate in anti-phase at twice the Rabi frequency. They settle at the same value, because $\hat{H}$ treats y and z symmetrically, hence converging towards a matching magnitude. The slight asymmetry from the relaxation is visually negligible here, due to its small numerical value of $\gamma_d = 0.1$ MHz.

The $\hat{\sigma}_x$ purity (blue dashed) stands apart — again through $[\hat{H}, \hat{\sigma}_x] = 0$ — showing both a pronounced early dip and the highest late plateau. The dip has the same origin as in the other bases: $|g\rangle$ carries maximal coherence across the x-axis, so early dephasing is strong. The repurification, however, is unique for this measurement basis: once the record has driven a trajectory towards the $\hat{\sigma}_x$ eigenbasis, only energy relaxation can't rotate it back off-axis (since the Hamiltonian does not affect the x-component of the Bloch vector), and it effectively regains some of its purity. It does not return to unity — the low measurement efficiency and the relaxation towards $|g\rangle$ prevent that — but the recovery leaves $\hat{\sigma}_x$ the purest of the three at late times (both for true and predicted). Regarding $\hat{\sigma}_y$ and $\hat{\sigma}_z$, the drive continuously rotates the state off the monitored axis, allowing for more dephasing, and the trajectories thus settle lower.

The reconstructed purities (red curves) sit systematically above the truth at their plateaus. The one noticeable exception is the early $\hat{\sigma}_x$ dip, which the network makes both deeper and earlier than the truth. Before the record reveals which eigenstate a trajectory is committing to, the network apparently benefits from significant mixing as a consequence of the uncertainty. Once the network has gathered sufficient information from the record (about which of the eigenstates the trajectory is approaching), it repurifies, even surpassing the truth due to the purity bias.

The small residual gap between the network's $\hat{\sigma}_y$ and $\hat{\sigma}_z$ plateaus do not seem to contain any physical content: the true dynamics fix the two to be equal, but nothing in the architecture enforces that symmetry, so it most likely reflects uneven training results by coincidence for the two different Kraus heads.

5 Discussion

5.1 Why choose an LSTM-Kraus structure?

Although this project demonstrates some of the advantages of the LSTM neural network structure (for instance predicting $\mathrm{Im}(\rho_{01})$ with some accuracy during measurement with $\hat{\sigma}_z$), there are several features of the LSTM structure that are not optimally used in this project. The primary limitation preventing the LSTM from truly outshining the conventional Bayesian filters (such as those described by Wonglakhon et al. [34]) in our case is the generation of multi-faceted experimental data. Examining the literature, we can see that particularly non-Markovian effects, rotation of the measurement axis and non-instant correlation of qubit and measurement result are said to pose a challenge to analytical filters, but can be handled by a neural network-type filter (see [1], [15]).

Koolstra et al. [15] show exactly where an LSTM network performs well by measuring on a *fast qubit*, i.e., a qubit where the Rabi frequency is faster than the cavity response time κ^{-1} distorting the measurement record in several ways.

During training they feed an LSTM network measurement records labelled by their projective measurement outcome taken at its stopping time T_m ($0 < T_m < 8\mu s$) and the network is trained by minimizing a Cross-Entropy (CE) loss function between the projective measurement and its prediction. Once trained, the LSTM is fed a heterodyne measurement record and outputs Bloch vector coordinates $\{x_k, y_k, z_k\}_{k=0}^{n}$ for the n timesteps in the measurement record.

Within Koolstra et al.'s fast regime ($2\Omega/\kappa > 1$), the photons are in the cavity for a time $\tau_c = 2/\kappa$ during which the qubit sweeps an arc in the yz-plane making the cavity output a time-average of a fraction of that Rabi cycle. Such a time-averaging effect makes the output photons depend not on the instantaneous qubit state but an average over recent states making the dynamics effectively non-Markovian ([15] sec. I). Such non-Markovian dynamics has the following effects and challenges:

- The time-averaging effect is equivalent to measuring the qubit along a rotated axis (towards the y-axis of the Bloch sphere) rather than the z-axis.
- Just as we perceived the two maximal outcomes as Gaussians with some separation: in [15] app. K.3 the time-averaging "[...] attenuates the eigenvalue contrast of $\hat{z}$ [...]", moving excited and ground states closer together. This also causes the measurement rate to effectively be reduced since the photons leaving the cavity represent a time-average, hence carrying less information about the qubit's current state. Thus, the measurement rate now depends on the Rabi frequency.

When the dynamics become fast ($2\Omega/\kappa > 1$) "[...] there is no choice of dt for which the standard Bayesian filter produces accurate trajectories" ([15] sec. III), so an analytical filter must be supplied with prior information such as Ω_R and κ: "This Bayesian filter method [...] [uses] a well-known initial state and calibrated values for Ω, the ensemble decay rate, and qubit-conditioned resonator output fields." ([15] sec. III). But Koolstra et al. show that this poses no problem to the LSTM which "[...] offers an accurate reconstruction method requiring no prior knowledge of coupling rates to the environment". So the LSTM does not need any specification of system or parameters to predict probabilities, which makes it ideal when parameters are time-dependent or cannot be calibrated. A particularly nice example of the LSTM network uncovering hidden time-dependencies is their introduction of a sinusoidal Rabi frequency within a trajectory. From the predicted LSTM trajectories, they can accurately recover the sinusoidal Rabi frequency and the subsequent correlation with measurement rate.

Conclusively, Koolstra et al. remarks that "[...] LSTM recurrent neural networks can outperform standard quantum filters, if the qubit dynamics are faster than the resonator linewidth." [15]. This shows promising

use-cases for recurrent neural network filters in regimes where Bayesian filters require assumptions and prior knowledge that might be partly inaccessible to the experimenters.

Koolstra et al.'s LSTM produces the set $\{x_k, y_k, z_k\}_{k=0}^{n}$ and their loss function contains 3 terms: the cross entropy, $\mathcal{L}_{CE}$; a "[...] mean-squared error for deviating from the known initial state at time $t = 0$, $\mathcal{L}_{init}$"; and $\mathcal{L}_{\text{purity}}$ than enforces that the produced Bloch vector does not lie outside the Bloch vector ([15] app. D.2 (i)-(iii)). The two latter act as soft physical constraints. However, one might also be interested in creating and evolving the full conditional state, $\rho(t)$, directly, which motivates the further employment of a physical map of a physical state, implemented as a Kraus operator-based decoder head, as described in section 2.5. This approach both has advantages and disadvantages:

Advantages.

- The Kraus formalism enables one to construct physically valid density matrices conditioned on a measurement record (see sec. 2.4), thus one does not need the additional loss components employed by Koolstra et al.
- Ensuring a physically motivated conditional forwarding of $\rho(t)$ through Kraus operators, interconnecting all ρ for a specific trajectory, also reduces the need for trajectories with variable duration. Our particular model might, however, gain additional precision from such an initiative (see sec. 5.2).
- A key component of using a Kraus-based forwarding is that the reconstructed object is a valid density matrix from which both the probabilities of a particular measurement (see eq. 2.9) and the behavior of the off-diagonal elements can be predicted.

Comparing to the literature, Singh & Bathia [30] has employed a similar scheme as in this thesis, but with emphasis on comparing Machine Learning models with and without the presence of a Kraus forwarding. Their model architecture and physical system are similar to the one applied in this thesis. Their LSTM backbone does, however, only feed the Kraus layer its hidden state (in contrast with this work, where $J(t)$, $\rho_{NN}(t)$ and h_t are all fed), which limits the comparison. Importantly, Singh & Bathia use supervision with the true $\rho(t)$ that they generated using QuTip. Their findings show that:"**Kraus-LSTM** achieves the highest absolute performance [...]"([30] p. 6). Being a supervised neural network, it requires the knowledge of a true $\rho(t)$, which we deliberately omitted to accommodate for the model being applicable to experimental data (an actual homodyne current), where no true $\rho(t)$ is accessible.

Disadvantages.

- One obvious disadvantage of a Kraus operator-based model is the added computation needed to sequentially forward $\rho_{\text{NN}}(t)$ and ensure QR-normalization for each time step. This extra computation might not be feasible when doing real-time quantum control.
- During training, the model creates a ρ_{NN}, and the end-of-trajectory BCE loss is back-propagated through the network to update the weights until $\partial\mathcal{L}/\partial W \to 0$. The LSTM network's output is not supervised directly, because at each step, its output has to be forwarded through the Kraus head and the non-linear QR normalization, and the resulting map for each $\rho(t)$ is applied recursively over the whole trajectory, so the loss depends on the LSTM output through a long, strongly nonlinear recursive chain. The gradient, $\partial\mathcal{L}/\partial W$, that the LSTM receives is a result of back-propagation through the whole chain, which can cause the gradient to vanish for early time steps. Conclusively, this makes the LSTM difficult to train.

5.2 Suggestions for improvements

In the Theory section, we describe how one could use Kraus operators to evolve a state after doing a measurement by conditioning one operator on the measurement and averaging over the rest. This procedure does not align exactly with the design of our model, in which we let the prediction of all four

Kraus operators depend on $J(t)$. To be completely true to the theory, we should have let only one of the four Kraus operators be conditioned on $J(t)$. In this way, we may have been too weakly committed to the physics part of the model. Thus, we imagine a format of Kraus operators that for a single time step evolves $\rho(t_k)$ in two steps: (1) an unconditional step independent of $J(t_k)$ and $h(t_k)$ as determined by 3 unconditional Kraus operators, where $\sum_{i=1}^{3} \hat{K}_i^\dagger \hat{K}_i = \hat{I}$ hence trace preserving; (2) a conditional step forwarded by a Kraus operator $\hat{M}$ dependent on both $J(t_k)$ and $h(t_k)$ representing the measurement action. $\rho(t_k)$ is thus forwarded in two steps to ensure correct normalization:

$$\tilde{\rho}(t_{k+1}) = \sum_{j=0}^{2} \hat{K}_j \, \rho(t_k) \, \hat{K}_j^\dagger, \qquad \sum_{j=0}^{2} \hat{K}_j^\dagger \hat{K}_j = \hat{I}, \tag{5.1}$$

$$\rho(t_{k+1}) = \frac{\hat{M}_k \, \tilde{\rho}(t_{k+1}) \, \hat{M}_k^\dagger}{\mathrm{Tr}\big[\hat{M}_k \, \tilde{\rho}(t_{k+1}) \, \hat{M}_k^\dagger\big]}, \qquad \hat{M}_k \equiv \hat{M}\big[J(t_k),\, h(t_k)\big], \tag{5.2}$$

Using this format, a greater distinction between conditional and unconditional dynamics could be made, thus one could imagine that the constructed Kraus operators would be more physical and perhaps resemble those that act on the system. We imagine the unconditional sum of Kraus operators to not receive an input and be updated through the loss function and the measurement record J(t) (and LSTM) to feed information into the conditional Kraus operator. Whether this would be the way that the neural network would physically realize the distinction between the Kraus operators is difficult to know, but assuming this form, the outputted Kraus operators can probably become recognizable.

Similarly, our chosen Kraus form probably impacted the predicted unconditional dynamics of the qubit, since we initialize the Kraus operators with the bias: $\hat{K}_0 = \hat{I},\ \hat{K}_1 = \hat{K}_2 = \hat{K}_3 = \hat{0}$. As discussed above, this had the unwanted consequence of creating too unitary dynamics. It was originally implemented to ensure that the predicted states did not contain dynamics at scales faster than the Hamiltonian — the primary driver of dynamics — being a major help, especially for the initial time steps in ρ, which are harder for the model to predict.

A suggestion that would fix the identity-thus-unitary initialization problem is to use a method similar to that of Koolstra et al. [15]. In their paper, the projective measurement is still performed once per trajectory, but it is done at different times during different trajectories, such that they effectively projectively measure at all times from $t = 0\,\mu s$ to $t = 8\,\mu s$, but in different trajectories. Doing this in the training phase could potentially improve its prediction for all times. This would of course constrain initial dynamics, but also those that happen further in the time series. Such an approach converges on full state-supervision of the neural network, but applied in a way that is experimentally viable.

Our findings show that the model most likely could not infer a per-trajectory Ω_R and adjusted its bias vector to include oscillations and thus reduce the loss function. It is hence something that the model found unconditionally, and not something that could be inferred per-trajectory. We suspect that this is due to the low SNR present in the trajectories, meaning that a Rabi frequency was simply not visible or at least not to a degree better than the $\pm 20\%$ variation in Ω_R used to generate trajectories. A straightforward way to make Ω_R more visible within the measurement current is to increase the time of observation — assuming that the oscillations are not too heavily damped at the end of the trajectory. One could also increase $\eta\kappa$, which would result in a stronger measurement. This would extract more information from the system, but it would also lead to higher backaction. Higher backaction leads to the measurement suppressing the unitary oscillation by the Hamiltonian, so tuning Ω_R in relation to $\eta\kappa$ is indeed a compromise.

Although the assumption behind initializing the qubit in the ground state $|g\rangle$ is that it is an energetically stable state, one could also imagine initializing the qubit along other axes of the Bloch sphere. We suspect that this would aid the model's prediction power in the y direction. When the qubit is initialized in $|g\rangle$, it is in a full superposition along the y-axis. A measurement instance with $\hat{\sigma}_y$ in this case would thus cause the maximal amount of back-action possible, since the information gain is high due to the uncertainty of the state. This would pose no problem if the efficiency was high — but since $\eta = 0.3$ most of the backaction

goes into an unmonitored channel thus making the $\mathrm{Im}(\rho_{01})$ prediction less reliable, initially. If we instead initiated the qubit in an eigenstate of $\hat{\sigma}_y$, the measurement backaction would initially be zero and increase over time, thus making the $\hat{\sigma}_y$ and $\hat{\sigma}_z$ measurements identical. This should of course not be implemented in the x-direction, since there is no Hamiltonian to drive the qubit away from such an eigenstate. This scheme would create an easier starting point for the model. One could also employ a more complicated scheme, such as Flurin et al. [7]: "Six preparations settings ($y_0 \in \{0,1\}$, $a \in \{X,Y,Z\}$) and six measurement settings ($y_T \in \{0,1\}$, $b \in \{X,Y,Z\}$) are used.", which would give the model a more varied training set.

Continuing the inspiration from Flurin et al. [7], another improvement is to implement the LSTM in both positive (forward) and negative (backward) time directions, thus making the LSTM network *bidirectional* (BiLSTM). In such a case, we imagine the BiLSTM obtaining hidden and cell state representations on a forward and backward pass of the measurement current and then concatenating the hidden state into a single forward-backward hidden state, which is fed to the Kraus decoder head responsible for forwarding ρ with this additional information and finally compute the loss and back-propagate. This is similar to methods within quantum smoothing [12], since the BiLSTM creates a hidden state representation during the forward and backward pass of the measurement current, similar to a forward and backward state created when using quantum smoothing. If one wanted to produce a forward-state (density) and a backward state, one could apply the formalism of Gammelmark et al. [9], where the backward object is an *effect matrix*. This object is analogous to a density matrix, but it propagates information backwards in time conditioned on future measurement outcomes. Of course, this is not something to be done in an online training scenario, but rather a scheme applied when the full measurement record is obtained, for example to enhance parameter estimation.

5.3 Mistake

5.3.1 Nonlinearity

Upon closer consideration, it becomes apparent that properly normalised conditioned maps (see eq. 2.10) are actually not linear due to the division by the trace (this claim of nonlinearity when monitoring is validated in [33] sec. "Feedback control of a monitored system"). Thus, the maps are not — strictly speaking — CPTP, since the term is pertained to linear maps (cf., [31] "Abstract"). This seemingly stands as an big oversight, since the entire network is said to be built around the CPTP assumption. On further inspection, this turns out to have a caveat as well: the network's Kraus operators depend on the same ρ that they act on (see Fig. 3.1), enabling non-linearity, which effectively still allows for reconstruction in accordance with the underlying physics. And CPTP or not, the reconstructed density matrices are — to high precision — both trace-preserved and positive semi-definite (see Figure 9.1), which was ultimately our initial goal of introducing CPTP.

One could additionally highlight that the primary objective and strength of the ML approach is that it presumably has better predictive power compared to analytical methods, and that the this gain in prediction power is what motivates an ML approach over an analytical approach in the first place.

Naturally, we would have liked have resolved this mistake, but being an error that entered the consciousness uncomfortably close to the deadline, the viable thing was to discuss the mistake here.

Conclusion and Perspectives

6

In this Bachelor Project, we have made a model consisting of an LSTM recurrent neural network and Kraus operators. Once trained, the model gives a prediction of the density matrix evolution $\rho(t)$ for a specific realisation of a Rabi qubit system, which has been monitored with weak homodyne measurement. It does so by forwarding $\rho_t \rightarrow \rho_{t+1}$ using four Kraus operators at each time step. This way of forwarding is, to high precision, trace-preserving and positive and consequently ensures that $\rho(t)$ stays an approximately physically valid density matrix. The model conditions its prediction on a record $J(t)$ resulting from measurement in one of the three $\hat{\sigma}_b$-directions, and a final projective measurement $y \in \{0, 1\}$ in the same direction. We use a BCE loss function, which optimizes the model's end prediction against the known projective measurement y.

The first variation of the model is trained on a total of $N = 30000$ trajectories spread evenly across the three measurement directions $\hat{\sigma}_b$ with a fixed Rabi frequency for all N. Quite naturally, the model makes the best predictions in the same direction as the one it is optimized against. That means the model predicts ρ_{00} best when $J(t)$ comes from from a $\langle\hat{\sigma}_z\rangle$ measurement; Re(ρ_{01}) best when $J(t)$ comes from from a $\langle\hat{\sigma}_x\rangle$ measurement; and Im(ρ_{01}) best when $J(t)$ comes from from a $\langle\hat{\sigma}_y\rangle$ measurement (when ρ is written in the z-basis). The performance decreases in the order mentioned. This is a result of a combined effect of the Rabi hamiltonian dynamics and the fact that the qubit is initialized in the state $|g\rangle$. Due to the physics restrictions enforced by the use of Kraus operators, the model still manages to predict the parts of $\rho(t)$ that is not directly present in the measurement signal $J(t)$ and which it is not optimized against. These predictions are generally biased towards purity-preserving evolution due the Kraus operators being initialized in a unitary way.

A second variation of the model is trained on trajectories with varying Rabi frequencies. The model does not manage to capture the varying frequencies of each trajectory but instead oscillates with the mean frequency in all cases. Our best explanation is that the signal-to-noise ratio in $J(t)$ is too low for the model to successfully extract the frequency: It takes more time for the model to gain enough information to resolve the state than one full Rabi period ($\tau_m > T_R$). Instead, it learns to exhibit oscillatory behaviour independently of the input, and this oscillation must therefore lie in the bias vectors.

Fitting the Stochastic Master Equation to ρ_{00}^{NN} for $\hat{C}_{\text{mon}} \propto \hat{\sigma}_z$ shows that the physical parameters used to generate the data cannot be accurately extracted from the model. The main problem is that the coherence elements are not accurately estimated, and a faithful parameter estimation requires a fit to the entire $\rho(t)$, which must in all aspects be sufficiently close to ρ_{true}. Another problem could be that we have — potentially erroneously — allowed the Kraus operators to be conditioned on ρ_{t-1}, which may make the forwarding depend on ρ in some non-linear way, and thus the predicted dynamics are not compatible with SME dynamics.

By averaging ρ_{00}, Re(ρ_{01}) and Im(ρ_{01}) across all trajectories, the Wiener-noise cancels, and we are left with the ensemble average dynamics, equal to the Lindblad evolution. Here one also sees that the predictions are best in the same direction as the measurement direction. The ensemble averages provide good qualitative understanding of which dynamics the model can and cannot 'see' from the measurement record.

As we have seen from literature and also sensed in our own humble investigation into the topic, Machine Learning approaches to quantum filtering have the potential to outperform Bayesian, analytical filters when exact parameter values and system dynamics are uncertain. However, to clearly see this quantum advantage, one would have to use real experimental data, and compare the prediction performance of a machine learning model to the best analytical prediction. A case where this would be especially relevant is the case of the fast qubit.

The advantages of using a physically motivated structure such as Kraus operators alongside a machine learning approach are physically valid states and as a consequence, a gain of additional state information, even for parts of ρ unmonitored. However, these additional predictions lack precision. Disadvantages of including a physically motivated structure is that it adds to the compute. One would have to investigate whether doing that extra computation in a real-time quantum control case is possible. It also might hurt the quality of predictions, since the model may have to deviate from the best prediction to ensure the physical validity.

We see our contribution as a physics-informed machine learning hybrid, which has the ability to produce physically valid states. We believe that tuning the model in the ways we describe in section 5.2 would aid in the model's predictions and thus one could imagine using a variant of this model on experimentally obtained data. One could imagine this being an advantage in cases, where one does not fully know the parameter values or the operators acting on the system.

Bibliography

7

[1] Leonardo Banchi, Edward Grant, Andrea Rocchetto, and Simone Severini. „Modelling Non-Markovian Quantum Processes with Recurrent Neural Networks“. In: *New Journal of Physics* 20.12 (Dec. 21, 2018), p. 123030. arXiv: 1808.01374[quant-ph].

[2] Colah's blog. *Understanding LSTM Networks – colah's blog*. May 1, 2026. URL: https://colah.github.io/posts/2015-08-Understanding-LSTMs/ (visited on May 1, 2026).

[3] PyTorch Docs. *AdamW — PyTorch 2.12 documentation*. May 28, 2026. URL: https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html (visited on May 28, 2026).

[4] PyTorch Docs. *CosineAnnealingLR — PyTorch 2.12 documentation*. May 28, 2026. URL: https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html (visited on May 28, 2026).

[5] PyTorch Docs. *Linear — PyTorch 2.11 documentation*. 2026. URL: https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html (visited on Apr. 28, 2026).

[6] PyTorch Docs. *PyTorch documentation — PyTorch 2.12 documentation*. May 24, 2026. URL: https://docs.pytorch.org/docs/stable/index.html (visited on May 24, 2026).

[7] E. Flurin, L. S. Martin, S. Hacohen-Gourgy, and I. Siddiqi. „Using a Recurrent Neural Network to Reconstruct Quantum Dynamics of a Superconducting Qubit from Physical Observations“. In: *Physical Review X* 10.1 (Jan. 9, 2020), p. 011006.

[8] Jay Gambetta, Alexandre Blais, M. Boissonneault, A. A. Houck, D. I. Schuster, and S. M. Girvin. „Quantum trajectory approach to circuit QED: Quantum jumps and the Zeno effect“. In: *Physical Review A* 77.1 (Jan. 25, 2008), p. 012112. arXiv: 0709.4264[cond-mat].

[9] Søren Gammelmark, Brian Julsgaard, and Klaus Mølmer. „Past quantum states“. In: *Physical Review Letters* 111.16 (Oct. 15, 2013), p. 160401. arXiv: 1305.0681[quant-ph].

[10] Guillaume Gennet and Christopher Bishop. „An Introduction to Useful Quantum Computing (by Alice & Bob)“. In: ().

[11] Élie Genois, Jonathan A. Gross, Agustin Di Paolo, Noah J. Stevenson, Gerwin Koolstra, Akel Hashim, Irfan Siddiqi, and Alexandre Blais. „Quantum-tailored machine-learning characterization of a superconducting qubit“. In: *PRX Quantum* 2.4 (Dec. 20, 2021), p. 040355. arXiv: 2106.13126[quant-ph].

[12] Ivonne Guevara and Howard Wiseman. „Quantum State Smoothing“. In: *Physical Review Letters* 115.18 (Oct. 30, 2015), p. 180407. arXiv: 1503.02799[quant-ph].

[13] Kurt Jacobs and Daniel A. Steck. „A straightforward introduction to continuous quantum measurement“. In: *Contemporary Physics* 47.5 (Sept. 2006), pp. 279–303.

[14] Andrew N. Jordan and Irfan A. Siddiqi. *Quantum Measurement: Theory and Practice*. 1st ed. Cambridge University Press, Feb. 15, 2024.

[15] G. Koolstra, N. Stevenson, S. Barzili, *et al.* „Monitoring Fast Superconducting Qubit Dynamics Using a Neural Network“. In: *Physical Review X* 12.3 (July 26, 2022), p. 031017.

[16] Anders Krogh and John A Hertz. „A Simple Weight Decay Can Improve Generalization“. In: ().

[17] Ben Kröse and Patrick van der Smagt. „Introduction to Neural Networks“. In: ().

[18] Neill Lambert, Eric Giguère, Paul Menczel, *et al.* „QuTiP 5: The Quantum Toolbox in Python“. In: *Physics Reports*. QuTiP 5: The Quantum Toolbox in Python 1153 (Jan. 18, 2026), pp. 1–62.

[19] IBM Quantum Learning. *Channel representations*. IBM Quantum Learning. June 8, 2026. URL: https://quantum.cloud.ibm.com/learning/en/courses/general-formulation-of-quantum-information/quantum-channels/quantum.cloud.ibm.com/learning/en/courses/general-formulation-of-quantum-information/quantum-channels/representations-of-channels (visited on June 8, 2026).

[20] IBM Quantum Learning. *Density matrix basics*. IBM Quantum Learning. June 7, 2026. URL: https://quantum.cloud.ibm.com/learning/en/courses/general-formulation-of-quantum-information/density-matrices/quantum.cloud.ibm.com/learning/en/courses/general-formulation-of-quantum-information/density-matrices/density-matrix-basics (visited on June 7, 2026).

[21] IBM Quantum Learning. *Fidelity*. IBM Quantum Learning. June 2, 2026. URL: https://quantum.cloud.ibm.com/learning/en/courses/general-formulation-of-quantum-information/purifications-and-fidelity/quantum.cloud.ibm.com/learning/en/courses/general-formulation-of-quantum-information/purifications-and-fidelity/fidelity (visited on June 2, 2026).

[22] William P. Livingston, Machiel S. Blok, Emmanuel Flurin, Justin Dressel, Andrew N. Jordan, and Irfan Siddiqi. „Experimental demonstration of continuous quantum error correction“. In: *Nature Communications* 13.1 (Apr. 28, 2022), p. 2307. arXiv: 2107.11398[quant-ph].

[23] Daniel Manzano. „A short introduction to the Lindblad master equation“. In: *AIP Advances* 10.2 (Feb. 1, 2020), p. 025106.

[24] Mahdi Naghiloo. *Introduction to Experimental Quantum Measurement with Superconducting Qubits*. Apr. 19, 2019. arXiv: 1904.09291[quant-ph].

[25] Michael A Nielsen and Isaac L Chuang. *Quantum Computation and Quantum Information*. Vol. 2010. Cambridge University Press, 2010.

[26] *Observable*. In: *Wikipedia*. Page Version ID: 1351217458. Apr. 26, 2026.

[27] PyTorch. *torch.linalg.qr — PyTorch 2.12 documentation*. URL: https://docs.pytorch.org/docs/stable/generated/torch.linalg.qr.html (visited on June 10, 2026).

[28] Qutip. *Stochastic Solver — QuTiP 5.0 Documentation*. URL: https://qutip.readthedocs.io/en/v5.0.1/guide/dynamics/dynamics-stochastic.html#stochastic-master-equation (visited on May 21, 2026).

[29] SciPy. *'L-BFGS-B' — SciPy v1.17.0 Manual*. SciPy Manual. June 5, 2026. URL: https://docs.scipy.org/doc/scipy/reference/optimize.minimize-lbfgsb.html#optimize-minimize-lbfgsb (visited on June 5, 2026).

[30] Priyanshi Singh and Krishna Bhatia. *Kraus Constrained Sequence Learning For Quantum Trajectories from Continuous Measurement*. Mar. 5, 2026. arXiv: 2603.05468[cs].

[31] Roderich Tumulka and Jonte Weixler. *Fixed Points of Completely Positive Trace-Preserving Maps in Infinite Dimension*. Nov. 22, 2024. arXiv: 2411.14800[math-ph].

[32] Wikipedia. "*Eigenvalues and characteristic polynomial*" *in 'Determinant'*. In: *Wikipedia*. Page Version ID: 1355713298. June 7, 2026.

[33] Howard M. Wiseman and Gerard J. Milburn. *Quantum measurement and control*. First paperback edition. Cambridge: Cambridge University Press, 2014. 460 pp.

[34] Nattaphong Wonglakhon, Areeya Chantasri, and Howard M. Wiseman. *Quantum trajectories for time-binned data and their closeness to fully conditioned quantum trajectories*. Version Number: 1. 2026.

Appendix A: LSTM Structure

8

As mentioned in 'Neural Networks and LSTM', the LSTM structure consists of a separate NN module for each temporal data point. Each module takes as input the current data value $\vec{x}_t$; the previous cell state value C_{t-1}; and the previous hidden state value $\vec{h}_{t-1}$. It computes the new cell state $\vec{C}_t$ and new hidden state $\vec{h}_t$ and passes it to the next module.

$$\vec{z}_t = [h_{t-1}, x_t] \tag{8.1}$$

Collects the input and previous hidden state in a single vector.

$$\vec{f}_t = \sigma(\mathbf{W}_f \vec{z}_t + \vec{b}_f) \tag{8.2}$$

Forget gate. Determines how much of the memory should come from earlier data. σ is the sigmoid function limiting the value to the range [0,1].

$$\vec{i}_t = \sigma(\mathbf{W}_i \vec{z}_t + \vec{b}_i) \tag{8.3}$$

Input gate. Decides how much the current data point should be stored in the cell state. A measure of how important the current data point is for the future evolution.

$$\tilde{C}_t = \tanh\left(\mathbf{W}_c \vec{z}_t + \vec{b}_c\right) \tag{8.4}$$

The candidate cell state. $\tanh$ limits the value to the range [-1,1].

$$\vec{C}_t = \vec{f}_t \odot \vec{C}_{t-1} + \vec{i}_t \odot \tilde{C}_t \tag{8.5}$$

The actual update of the cell state that is passed to the next module. $\odot$ is element-wise multiplication (the Hadamard product).

$$\vec{O}_t = \sigma(\mathbf{W}_o \vec{z}_t + \vec{b}_o) \tag{8.6}$$

The output gate. Used to determine the hidden state.

$$\vec{h}_t = \vec{O}_t \odot \tanh \vec{C}_t \tag{8.7}$$

Computing the new hidden state. $\tanh \vec{C}_t$ gives a scaled version of the memory.

The dimension of the hidden state and cell state is a design choice. Furthermore, each module can consist of multiple layers of the above gate functions.

The source used in this appendix is [2].

9 Appendix B: Physical Validity of the Reconstructed Density Matrices

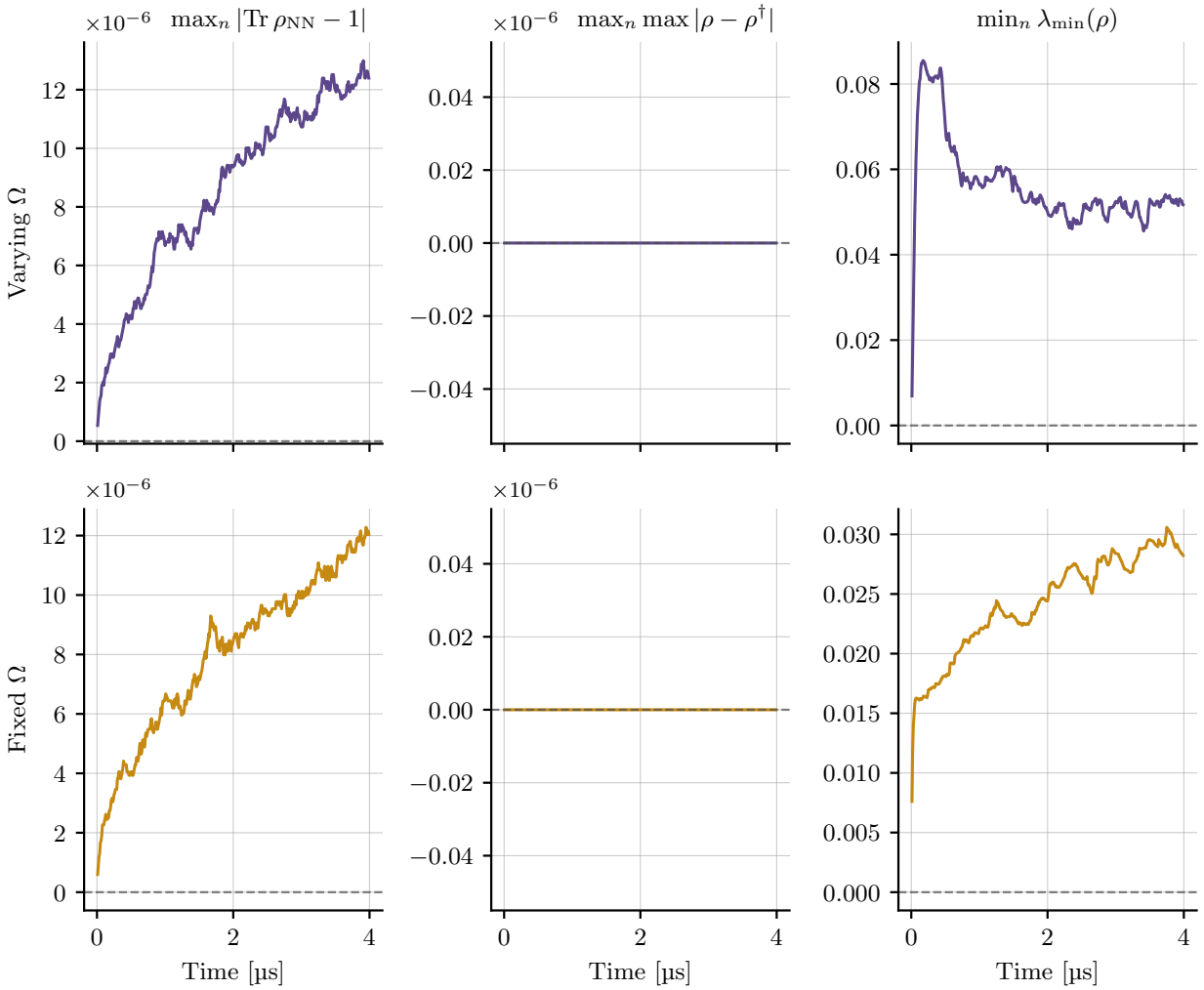

Figure 9.1: Physical-validity diagnostics for the reconstructed density matrices, evaluated at every time step over the 10,000 evaluation trajectories obtained from the SME data measured and projected onto the z-basis. Together, the three diagnostics test whether each reconstruction is a valid density operator. The top row shows the resulting diagnostics for the varying-Rabi dataset (purple curves), whereas the bottom row displays it for the single fixed Rabi frequency dataset (yellow curves). The columns report the deviation from unit trace (left); the Hermiticity error (middle); and the minimum eigenvalue (right). At each time step only the worst-case (maximum deviation from 0) value across the 10,000 trajectories is plotted. Dashed horizontal lines mark the zero reference.

10 Appendix C: Reconstruction of the Density Matrix for x and y

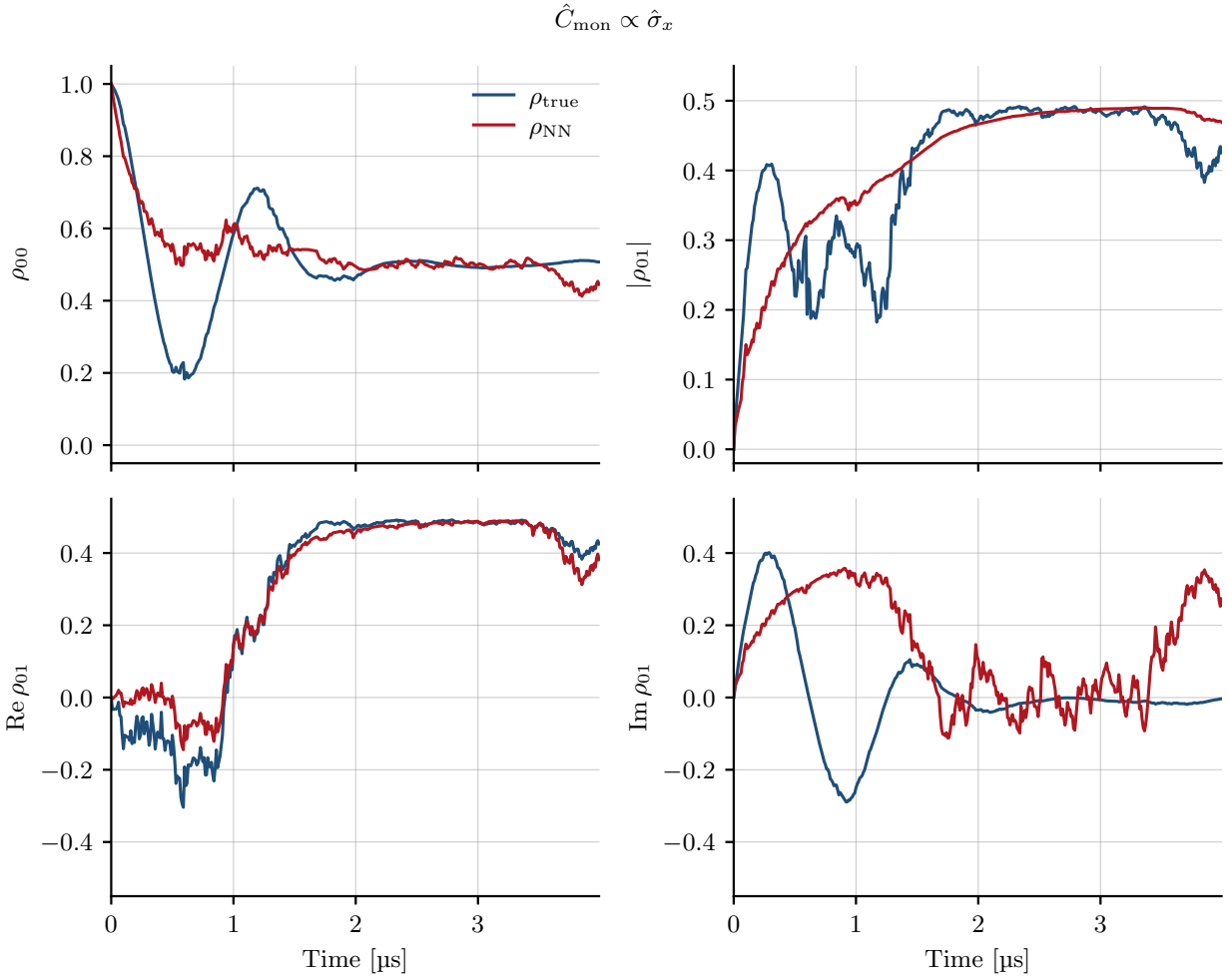

Figure 10.1: Reconstruction of the density matrix $\rho(t)$ by the model for a specific realisation, trained and evaluated on x-basis measurement records at a single fixed Rabi frequency, and informed by a final projective measurement in that same basis. The panels show the population ρ_{00}, the coherence magnitude $|\rho_{01}|$, and its real and imaginary parts $\mathrm{Re}(\rho_{01})$ and $\mathrm{Im}(\rho_{01})$, all expressed in the z-basis. Blue: ground truth ρ_{true} from `smesolve`; red: model reconstruction ρ_{NN}.

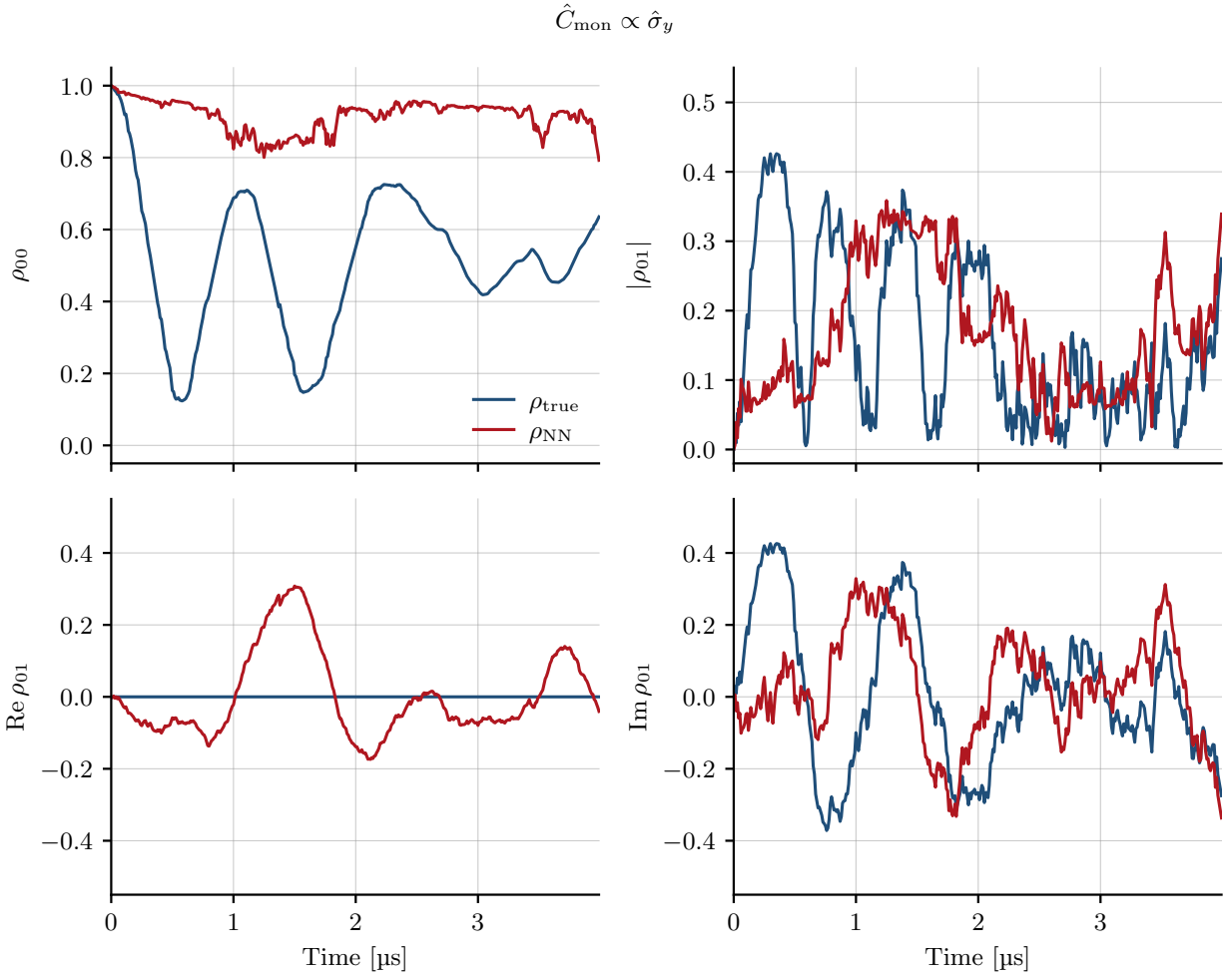

Figure 10.2: Reconstruction of the density matrix $\rho(t)$ by the model for a specific realisation, trained and evaluated on y-basis measurement records at a single fixed Rabi frequency, and informed by a final projective measurement in that same basis. The panels show the population ρ_{00}, the coherence magnitude $|\rho_{01}|$, and its real and imaginary parts $\mathrm{Re}(\rho_{01})$ and $\mathrm{Im}(\rho_{01})$, all expressed in the z-basis. Blue: ground truth ρ_{true} from `smesolve`; red: model reconstruction ρ_{NN}.

Appendix D: End Prediction

11

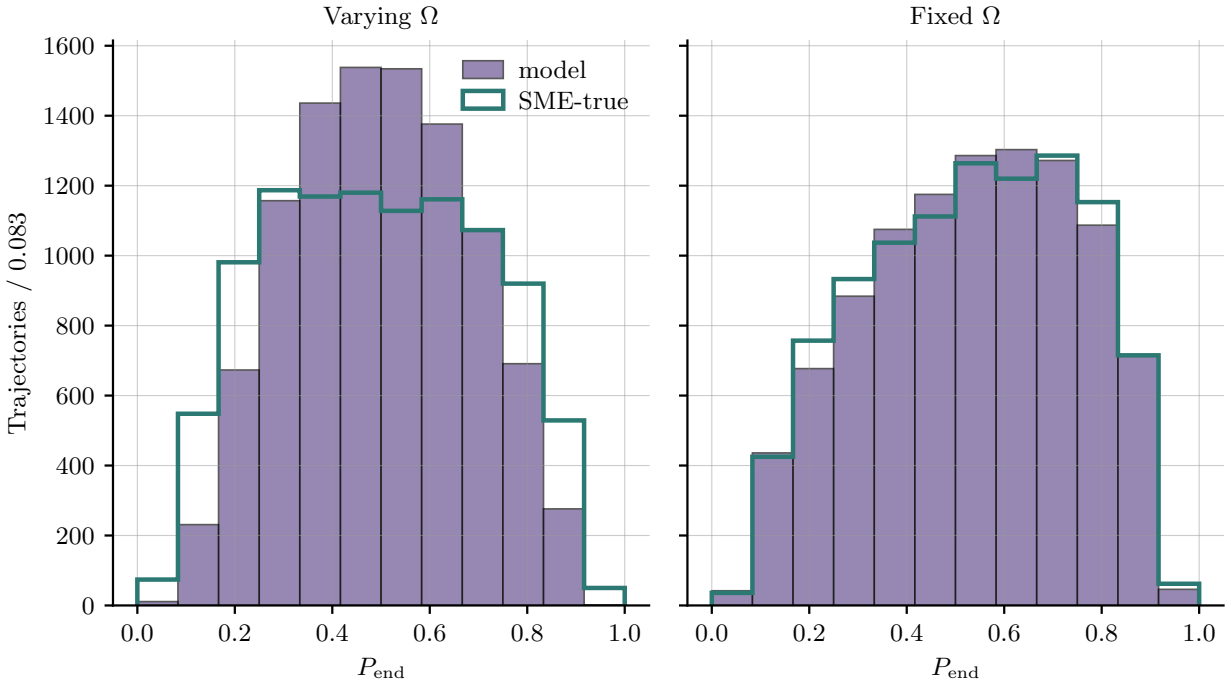

Figure 11.1: Histogram of the predicted final population P_{end} in the $\hat{\sigma}_z$ basis. The distribution of P_{end} over trajectories for the network (purple, filled) and for the true value propagated directly from the `SME` data (green, outline). Shown for both the varying and fixed Ω_R model.

Deklaration for anvendelse af generative AI-værktøjer (studerende)

På Københavns Universitet udfører vi vores arbejde med **ansvarsfølelse** og **respekt for samfund, kulturarv, miljø og mennesker** omkring os.
Integritet, ærlighed og **transparens** er forudsætninger for akademisk arbejde. Vi forventer derfor, at eksamenspræstationer afspejler **den studerendes egen læring og selvstændige indsats**.
Akademisk arbejde baserer sig altid på andres indsigter, viden og bidrag, men altid med **grundig anerkendelse, respekt** og **kreditering** af dette arbejde.
Dette gælder også ved brug af generativ kunstig intelligens.

Vejledning

Brug af generativ AI ved eksamen

I henhold til KU's regler for brugen af værktøjer, der er baseret på generativ AI (GAI), skal du være transparent om din anvendelse af teknologien, fx i dit metodeafsnit og/eller ved at udfylde og vedlægge nedenstående deklarationsskabelon som bilag til skriftlige opgavebesvarelser.
Når du skriver din deklaration, er det vigtigt, at læseren får et tydeligt billede af, om og hvordan generativ AI har bidraget til det endelige produkt.

Hvis det er besluttet, at du skal bruge skabelonen i dit fag, skal du også benytte den, når du *ikke* har anvendt GAI-værktøjer som hjælpemiddel. I dette tilfælde skal du dog blot krydse af, at du ikke har brugt GAI, og behøver ikke at udfylde resten.
Ved at deklarere din brug af GAI-værktøjer sikrer du, at der ikke opstår udfordringer i forhold til reglerne om eksamenssnyd.

I kurser, hvor brugen af GAI er integreret i fagligheden, kan refleksion og kritisk vurdering af anvendelsen af GAI-værktøjer også indgå som en del af et metodeafsnit i din opgave. Spørg din underviser eller vejleder, hvis du er tvivl, om det er tilfældet i dit kursus.

Hvis generativ AI er objekt for din undersøgelse, vil det fremgå af dine forskningsspørgsmål, din metodebeskrivelse, din analyse og konklusion, hvilken rolle GAI spiller i din opgave. Hvis du

samtidig også bruger GAI som hjælpemiddel i processen, skal du deklarere denne anvendelse særskilt.

Opmærksomhedspunkter:

- Hvis GAI er et tilladt hjælpemiddel på dit kursus, må du anvende GAI til dialog og sparring under udarbejdelsen af din opgave, men du må **ikke** overlade udfærdigelsen af din opgavebesvarelse til GAI-værktøjer, selvom alle hjælpemidler er tilladt.
- Hvis materiale fra GAI inkluderes som kilde (direkte eller i redigeret form) i din besvarelse, gælder de samme krav om brug af citationstegn og kildehenvisning som ved alle andre kilder, da der ellers vil være tale om plagiat.
- Brug aldrig personhenførbare, ophavsretsbeskyttede eller fortrolige data i et AI-værktøj.
- Husk altid at undersøge gældende regler og retningslinjer for brug af generativ AI på KU.
- Læs kursusbeskrivelsen grundigt. Det er vigtigt at du ved, hvilke anvendelser der er tilladt i dit kursus. Der kan eventuelt være yderligere krav om dokumentation, fx at du skal beskrive dine centrale prompts og evt. kildemateriale (hvad har du givet af kontekst, hvad har du fodret værktøjet med, hvad har du bedt værktøjet om at gøre), beskrive outputtet (hvilke svar du fik af værktøjet), beskrive processen, f.eks. historik og iterationer (hvis du har skrevet frem og tilbage med værktøjet ad flere omgange for at komme frem til et brugbart svar).
- Tal med din underviser eller vejleder, hvis du er i tvivl.

Deklaration for anvendelse af generative AI-værktøjer (studerende)

☒ **Jeg/vi har benyttet generativ AI som hjælpemiddel/værktøj** *(sæt kryds)*

☐ **Jeg/vi har IKKE benyttet generativ AI som hjælpemiddel/værktøj** *(sæt kryds)*

Hvis brug af generativ AI er tilladt til eksamen, men du ikke har benyttet det i din opgave, skal du blot krydse af, at du ikke har brugt GAI, og behøver ikke at udfylde resten.

Oplist, hvilke GAI-værktøjer der er benyttet, inkl. link til platformen (hvis muligt):

Claude (Model Opus 4.6) https://claude.ai/

Beskriv hvordan generativ AI er anvendt i opgaven:

1) ***Formål (hvad har du/I brugt værktøjet til)***
   Vi har brugt GAI som hjælp til kodeprocessen. Desuden har vi brugt GAI til litteratursøgning, altså som hjælp til at finde relevante artikler inden for feltet. Vi har også benyttet den til uddybende forstående spørgsmål, dog aldrig som eksplicit kilde. Vi har også benyttet den som værktøj til at lave grafiske skemaer ved at lave en beskrivelse af den ønskede figur, som så ved GAI er blevet oversat til kode i latex.

2) ***Arbejdsfase (hvornår i arbejdsprocessen har du/I brugt GAI)***
   Jævnt fordelt under hele projektet, eftersom vi har kodet under det meste af projektet.

3) ***Hvad gjorde du/I med outputtet (herunder også, om du/I har redigeret outputtet og arbejdet videre med det)***

Outputtet indgår i vores kode (som kan findes i GitHub repository). Desuden de grafiske skemaer i latex. Ellers er outputtet ikke gengivet i projektet.

NB. GAI-genereret indhold brugt som kilde i opgaven kræver korrekt brug af citationstegn og kildehenvisning. Læs retningslinjer fra Københavns Universitetsbibliotek på KUnet.